

STŘEDOŠKOLSKÁ ODBORNÁ ČINNOST

Obor č. 18: Informatika

EMULACE HERNÍ KONZOLE NES NA MIKROKONTROLÉRU RP2350

EMULATION OF THE NES VIDEO GAME CONSOLE ON AN RP2350 MICROCONTROLLER

AUTOŘI	Alexey Kachaev, Jakub Jansa
ŠKOLA	Střední průmyslová škola elektrotechnická, Praha 2, Ječná 30
KRAJ	Pražský
KONZULTANT	Ing. Dušan Kuchařík
OBOR	18. Informatika
ROK	2026

Prohlášení

Prohlašujeme, že jsme naši práci SOČ vypracovali samostatně a použili jsme pouze prameny a literaturu uvedené v seznamu bibliografických záznamů.

Bereme na vědomí, že nejpozději odevzdáním slovesné vědecké práce do veřejné soutěže Středošolská odborná činnost, stejně jako odevzdáním jejích příloh a dalších připojených děl, např. audiovizuálních, fotografických, výtvarných, architektonických apod. (dále jen „soutěžní dílo“), dochází ke zveřejnění díla podle § 4 odst. 1 zákona č. 121/2000 Sb., autorského zákona, ve znění pozdějších předpisů (dále jen „autorský zákon“). Totéž platí pro pozdější odevzdání doplněného, změněného, upraveného nebo opraveného díla.

Bereme na vědomí, že zveřejněním díla, jehož součástí je vynález, se tento vynález stává součástí stavu techniky podle § 5 odst. 1, 2 zákona č. 527/1990 Sb., o vynálezech, průmyslových vzorech a zlepšovacích návrzích, ve znění pozdějších předpisů (dále jen „patentový zákon“), což zakládá překážku pro udělení patentu podle § 3 odst. 1 patentového zákona.

Bereme na vědomí, že vyhlášovatel soutěže je podle § 61 odst. 1 autorského zákona per analogiam oprávněn užít soutěžní dílo pro účely zajištění průběhu soutěže, zejména k zajištění transparentnosti soutěže a veřejnosti obhajob soutěžních prací. V odůvodněném rozsahu je tedy vyhlášovatel po dobu účasti autorů v soutěži oprávněn zejména:

- zhotovovat rozmnoženiny díla, je-li to nezbytné k seznámení účastníků soutěže, porotců nebo veřejnosti se soutěžní prací;
- zapůjčit originál nebo rozmnoženinu díla účastníkům soutěže, porotcům nebo veřejnosti. Přitom dbá na bezpečné nakládání s dílem;
- vystavovat originál nebo rozmnoženinu díla v průběhu soutěžních přehlídek a doprovodných akcí;
- sdělovat dílo veřejnosti v nehmotné podobě, a to především počítačovou nebo obdobnou sítí.

Dále prohlašujeme, že při tvorbě této práce jsme použili následující nástroje generativního modelu AI:

- ChatGPT (<https://chatgpt.com/>) – kontrola gramatiky, mírná úprava textu se zachováním autorských myšlenek a stylistiky.
- Google Gemini (<https://gemini.google.com/>) – kontrola gramatiky, mírná úprava textu se zachováním autorských myšlenek a stylistiky.

Po použití těchto nástrojů jsme provedli kontrolu obsahu a přebíráme za něj plnou zodpovědnost.

V Praze dne 14. května 2026: _____

Poděkování

Velice bychom chtěli poděkovat našemu konzultantovi Ing. Dušanovi Kuchařikovi za vedení naší práce a za podporu. Také chceme poděkovat Bc. Adamovi Papulovi za poskytnutí šablony pro naši prezentaci v sázecím systému $\text{\LaTeX}$ a také kontrolu čitelností práce. Dále velice rádi bychom poděkovali Ing. Janě Exnerové za zájem o naši práci a také poskytnutí teoretického materiálu o operačních zesilovačích. Také děkujeme našim rodičům a blízkým lidem za morální a finanční podporu.

Anotace

Tento projekt se zabývá problematikou nízkoúrovňové emulace historických výpočetních systémů, přičemž jako primárním předmětem studie slouží herní konzole Nintendo Entertainment System (NES). Práce analyzuje vnitřní architekturu procesoru R2A03 (odvozený od MOS 6502), grafického subsystému PPU a zvukové jednotky APU, a to s odkazem na obecnou teorii hardwarové emulace. V praktické části práce se zabývá návrhem a realizací emulační platformy na moderním mikrokontroléru RP2350 (Raspberry Pi Pico 2). Na rozdíl od běžných softwarových emulátorů běžících v prostředí plnohodnotného operačního systému je předložené řešení koncipováno jako aplikace typu bare-metal, tedy bez jakékoli abstraktní systémové vrstvy. Výsledkem je kompaktní, samostatné zařízení s nízkými hardwarovými nároky, které díky podpoře standardizovaných rozhraní pro přenos obrazu a zvuku (HDMI a analogový audio výstup) je přímo použitelné v běžném domácím prostředí.

Klíčová slova

Embedded; emulátor; mikrokontrolér; NES; počítačová architektura.

Annotation

This project addresses the issue of low-level emulation of historical computing systems, with the Nintendo Entertainment System (NES) game console serving as the primary subject of study. This paper systematically analyzes the internal architecture of the R2A03 processor (derived from the MOS 6502), the PPU graphics subsystem, and the APU sound unit, with reference to the general theory of hardware emulation. The practical part is dedicated to the design and implementation of an emulation platform on the modern RP2350 microcontroller (Raspberry Pi Pico 2). Unlike conventional software emulators running in a full-fledged operating system environment, the presented solution is designed as a bare-metal application, i.e., without any abstract system layer. The result is a compact, standalone device with low hardware requirements that, thanks to support for standardized interfaces for video and audio transmission (HDMI and analog audio output), is directly usable in a typical home environment.

Keywords

Computer architecture; embedded; emulator; microcontroller; NES.

Obsah

Použité technologie	9
Úvod	10
Právní aspekty emulace	11
1 Emulace počítačového systému	12
1.1 Úroveň abstrakce a přesnost	12
1.2 Skutečný čas a emulovaný čas	13
1.3 Metoda dohánění (catch-up)	13
1.4 Emulace CPU	15
1.5 Emulace pamětí a sběrnice	16
1.5.1 Přepínání banek	17
1.5.2 I/O a vedlejší efekty (Side Effects)	17
1.6 Dlaždicová grafika	17
1.6.1 Softwarové přístupy k vykreslování v emulátoru	18
1.7 Emulace zvuku	19
1.7.1 Syntéza zvuku	19
1.7.2 Vzorkovací (sampled) audio	20
1.7.3 Resampling a aliasing	20
2 Architektura NES	21
2.1 Herní kazety a správa paměti (Mappers)	21
2.1.1 Struktura herní kazety a ROM obraz	21
2.1.2 Formát iNES	22
2.1.3 Mechanismy bankování paměti (Mappers)	22
2.2 CPU	22
2.2.1 Architektura registrů	23
2.2.2 Paměťová organizace a adresování	23
2.2.3 Přerušení	24
2.2.4 Adresovací režimy	24
2.2.5 Neoficiální instrukce	24
2.3 PPU	25

2.3.1	Paměťová organizace a sběrnice	25
2.3.2	Vystavené registry	25
2.3.3	Vzory dlaždic a tabulky vzorů (Pattern Tables)	26
2.3.4	Jmenné tabulky (Nametables)	26
2.3.5	Tabulky atributů (Attribute Tables) a palety	27
2.3.6	Objekty a sprity (OAM)	27
2.3.7	Evaluační sprity (prohledávání OAM)	27
2.3.8	Sprite overflow	27
2.3.9	Scrolling	28
2.3.10	Proces vykreslování a časování	29
2.4	APU	29
2.4.1	Časování	29
2.4.2	Struktura kanálů	30
2.4.3	Snímkovací čítač (Frame Counter)	31
2.4.4	Mixér a výstup	31
3	Specifikace požadavků	32
3.1	Software	32
3.2	Hardware	33
4	Architektura projektu	34
5	Struktura zdrojového kódu	36
6	Návrh emulátoru	37
6.0.1	Smyčka emulátoru	37
6.1	ROM obrazy a mappery	38
6.2	Sběrnice	38
6.3	CPU	39
6.3.1	Vykonávací středisko CPU	39
6.4	PPU	40
6.4.1	Vykreslování pozadí	41
6.4.2	Příprava dat pro vykreslování pozadí	42
6.4.3	Vykreslování spritů	42
6.4.4	Příprava dat pro vykreslování spritů	43
6.4.5	Prohledávání OAM (Sprite Evaluation)	43
6.5	APU	45
6.5.1	Vzorkování (sampling)	45
6.5.2	Krokování a generování vzorků	46
6.6	BIOS	46
6.7	FWX (FirmWare eXchange)	49

6.7.1	Iniciační a terminační sekvence (START/STOP)	50
6.7.2	Přenos dat a příkazů	50
6.7.3	Zpracování příkazu (blokující exekuce)	50
6.7.4	Příjem návratových dat	51
6.7.5	Řetězení příkazů	51
6.8	Mapování a přehled registrů	51
6.8.1	Detailní popis registrů	52
6.9	Sada příkazu	54
6.9.1	Přehled podporovaných příkazů	54
7	Návrh základní desky	56
7.1	Napájení	57
7.2	Video výstup HDMI	57
7.3	Rozhraní SD karty	57
7.4	Analogový audio výstup	58
7.4.1	Rekonstrukce a předzesílení	58
7.4.2	Výkonové zesílení a výstup	58
7.5	Mikrokontrolér	59
8	Návrh ovladačů	60
9	Návrh mechanických krytů	61
9.1	Kryt základní desky	61
9.2	Kryt ovladače	61
9.3	Výroba	62
10	Návrh firmwaru	63
10.1	Generace video výstupu DVI	64
10.2	Generace audio výstupu	65
10.2.1	Konfigurace PWM	65
10.2.2	Architektura DMA přenosu	66
10.2.3	Konfigurace DMA	68
10.2.4	Generace vzorků	68
10.3	Obsluha SD karty	68
10.4	Čtení vstupu z ovladačů	69
10.4.1	Implementace	70
11	Testování a ověření správnosti	71
11.1	Testování CPU	71
11.2	Komplexní testování	71
11.3	Benchmarking a profilování	73
11.4	Vlastní ladicí nástroje	74

11.5 Závěr	75
12 Závěr	76
Použitá literatura	78
Seznam výpisů	80
Seznam obrázků	81
Seznam tabulek	83
Seznam příloh	84

Použité technologie

Vzhledem k rozsahu projektu bylo využito široké spektrum technologií od nízkoúrovňového programování až po mechanický návrh. Výběr nástrojů reflektoval technické požadavky, dostupnost dokumentace a také preferenci otevřeného softwaru (*FOSS*). Klíčové technologie jsou shrnuty v tabulce 1.

Kategorie	Technologie	Využití v projektu
Vývoj a sestavení	Jazyk C, GCC	Hlavní implementační jazyk a kompilátor pro architekturu ARM (RP2350).
	CMake	Automatizace sestavení, správa závislostí a multiplatformní podpora.
	Git	Správa verzí, historie změn a synchronizace zdrojového kódu.
Emulace a BIOS	cc65	Toolchain pro 6502; sestavení BIOSu a testovacích programů.
	neslib	Vývoj UI BIOSu (práce s registry PPU, správa palet a textu).
	Unity	Unit testing framework pro validaci implementace instrukcí CPU 6502.
Embedded systém	Pico SDK	Oficiální abstrakční vrstva (HAL) pro nízkoúrovňové řízení RP2350.
	PicoDVI	Softwarové generování DVI/HDMI signálu pomocí PIO a DMA.
	FatFS	Implementace souborového systému FAT32 pro načítání dat z SD karty.
Hardware a kryty	KiCad	Návrh schémat a desky plošných spojů (PCB) emulátoru.
	Fusion 360	Konstrukce a 3D modelování mechanické schránky zařízení.
	PrusaSlicer	Příprava dat (G-code) pro 3D tisk součástí krytu.
Desktop backend	SDL3	Abstrakce grafiky, zvuku a vstupů pro ladění emulačního jádra na PC.
	C++, ImGui	Vývoj vlastních ladicích nástrojů.
Dokumentace	L ^A T _E X	Sazba textu této práce a prezentace.
	LuaL ^A T _E X	Kompilátor zdrojových souborů textové práce v sázecím systému L ^A T _E X.
	PlantUML	Kreslení časových diagramů.
	draw.io	Kreslení diagramů a ilustrací.

Tabulka 1: Přehled použitých technologií, knihoven a nástrojů.

Úvod

Porozumění vnitřnímu fungování počítačových systémů patří mezi základní cíle výuky informatiky a elektrotechniky. Moderní počítače jsou však charakterizovány vysokou mírou komplexity a mnoha vrstvami abstrakce, které skryjí základní principy interakce hardwaru a softwaru.

Osvědčeným přístupem k této problematice je **emulace historických výpočetních systémů**. Jejich architektura je srovnatelně kompaktní, dobře zdokumentovaná a zároveň nabízí dostatek zajímavých technických výzev. Emulace umožňuje studovat chování systému a interakci jeho komponent, aniž by bylo nutné disponovat původním hardwarem, a zároveň poskytuje prostor pro experimentování a analýzu na různých úrovních abstrakce.

Většina existujících emulátorů je však realizována jako **softwarové aplikace pro osobní počítače nebo mobilní telefony**. Taková řešení sice nabízejí vysoký výkon, ale při vývoji bývá ignorována řada klíčových aspektů návrhu reálných systémů. Emulátor běžící pod desktopovým operačním systémem nemá přímou kontrolu nad přesným časováním, vstupně-výstupními zařízeními ani generováním signálů. Tyto úlohy jsou abstrahovány ovladači a knihovnamí, což vývojáře zbavuje nutnosti porozumět fyzické podstatě generovaných signálů a řešit kritické aspekty deterministického řízení hardwaru.

Tato práce si klade za cíl **propojit svět emulace a embedded systémů** do uceleného celku. Jako referenční architektura byl zvolen **Nintendo Entertainment System (NES)**, který představuje významný příklad kompletního systému s procesorem, grafickým a zvukovým subsystémem. Výsledkem práce je návrh a realizace **samostatné emulační platformy** na mikrokontroléru **RP2350 (Raspberry Pi Pico 2)**. Zařízení funguje bez operačního systému (*bare-metal*) s minimálním využitím knihoven třetích stran, přičemž je přímo připojitelné k běžným monitorům a reproduktorům.

Je důležité poznamenat, že původní hry pro systém NES zde slouží primárně jako **demonstrační prvek** funkčnosti a prostředek k vysvětlení technických konceptů. Získané znalosti o nízkoúrovňovém programování a řízení periférií jsou však univerzálně aplikovatelné i u jiných platform, a to nejen herních konzol.

Právní aspekty emulace

Vývoj softwarových emulátorů se z legislativního hlediska často pohybuje na hranici tzv. šedé zóny. Je nutné důsledně rozlišovat mezi samotným emulátorem, který představuje pouze nezávislou softwarovou implementaci hardwarové architektury, a autorsky chráněným obsahem, jakým jsou herní soubory (ROM obrazy) nebo případně původní chráněný BIOS. Zatímco tvorba a šíření emulátorů pro moderní, stále komerčně aktivní a vyráběné systémy naráží na silný právní odpor držitelů autorských práv, v případě historických konzolí je situace odlišná. Výroba platformy Nintendo Entertainment System (NES) byla ukončena před desítkami let. Samotný vývoj emulátoru dnes spadá primárně do oblasti studia výpočetní techniky a zachování digitální historie (*digital preservation*). Veškeré technické specifikace využitě v této práci byly získány výhradně z veřejně dostupné komunitní dokumentace na internetu nebo studiím chování již existujících emulátorů. Žádná část zdrojového kódu nepochází z uniklých proprietárních materiálů.

Veškeré komerční herní tituly, které byly v rámci vývoje spuštěny nebo jsou zmíněny v textu práce, případně použité pro demonstraci při obhajobě, sloužily výhradně jako nezbytný testovací a demonstrační prvek pro ověření přesnosti a správnosti emulovaného hardwaru. Práce vznikla za čistě edukativními a výzkumnými účely. Veškerá autorská práva i ochranné známky spojené s těmito hrami plně náleží jejich oprávněným vlastníkům a vydavatelům, zejména pak společnosti Nintendo a příslušným vývojářským studiím.

Tato práce ani její autoři v žádném ohledu nepodporují nelegální distribuci, sdílení či stahování autorsky chráněných ROM obrazů. Projekt je koncipován s předpokladem, že uživatel disponuje legálně nabytými daty, získanými například vytvořením digitální zálohy (*dumpingem*) z vlastnoručně zakoupených fyzických paměťových kazet.

Kapitola 1

Emulace počítačového systému

Emulace představuje proces, při němž jeden systém (**hostitelský systém**) napodobuje architekturu a chování jiného systému (**cílový systém**). Cílem není pouze spuštění programu, ale zachování jeho pozorovatelného chování tak, aby software původně určený pro cílovou architekturu fungoval bez modifikací (binární kompatibilita). Emulace tedy realizuje instrukční sadu, stav procesoru, paměťový model i komunikaci s periferiemi systému.

Prostředí, ve kterém je napodobovaná architektura, se nazývá **emulátorem**. Emulátor může být realizován jak softwarově (program), tak i hardwarově (FPGA nebo obvod z logických hradel). Tato kapitola bude rozebírat primárně softwarový přístup.

1.1 Úroveň abstrakce a přesnost

Při návrhu emulátoru je nutné rozhodnout, na jaké úrovni abstrakce bude cílový systém modelován. Obecně lze rozlišit dva základní přístupy:[1]

- **Vysokoúrovňová emulace** (*High-Level Emulation – HLE*) – soustředí se na reprodukci funkčního chování systému, často bez přesného modelování jednotlivých komponentů. Některé operace mohou být nahrazeny přímým voláním funkcí hostitelského systému. Tento přístup je obvykle rychlejší, ale může vést k omezené přesnosti a kompatibilitě.
- **Nízkoúrovňová emulace** (*Low-Level Emulation – LLE*) – snaží se modelovat architekturu co nejvěrněji, včetně registrů, instrukční sady, paměťové mapy a časování. Tento přístup je výpočetně náročnější, ale poskytuje vyšší přesnost a kompatibilitu.

Přesnost emulace vyjadřuje do jaké míry emulátor napodobuje chování původní architektury. Základním kritériem správnosti emulátoru je zachování pozorovatelného chování systému, tedy reakce na vstupy a generované výstupy. Emulátor lze považovat za správný tehdy, pokud pro každou posloupnost vstupů produkuje stejnou posloupnost výstupů jako původní systém. To zahrnuje nejen logickou správnost instrukcí, ale často i jejich časové vztahy.

Vyšší úroveň přesnosti znamená vyšší nároky na výkon hostitelského systému. Obvykle méně přesné emulátory preferují výkon hostitelského systému a jednoduchost návrhu. Více přesné emulátory preferují přesnost emulace cenou výkonu. Emulátory střední přesnosti představují zlatou střední mezi nízkou a vysokou přesností.

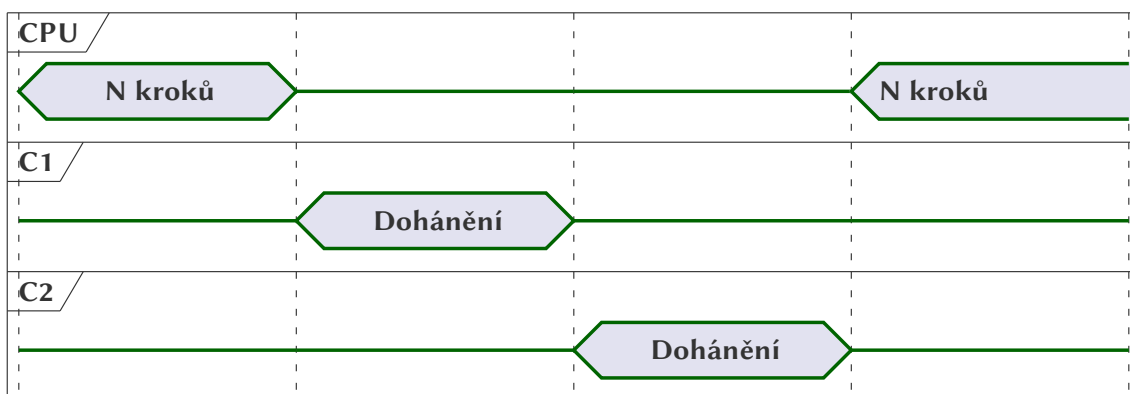
1.2 Skutečný čas a emulovaný čas

Pro dosažení precizní synchronizace zavedeme koncept cyklové osy, na které jsou vyneseny všechny emulované komponenty systému. Tato osa představuje společné měřítko, kde každý bod odpovídá konkrétnímu stavu všech subsystémů současně, přičemž její jednotkou jsou synchronizační body. V těchto bodech dochází k předání řízení mezi jednotlivými komponentami; jeden komponent vykonávání pozastaví a navazuje na něj další, tak aby byl zachován správný sled operací.

Díky tomu lze přesně modelovat vzájemné interakce, například moment, kdy grafický čip přistupuje ke sdílené paměti v přesně definovaném cyklu procesoru. Emulátor tak nepracuje s reálným časem, ale s „virtuálními tiky“, které zaručují, že operace proběhnou ve stejném pořadí a se stejným relativním časováním jako na originálním hardwaru.

1.3 Metoda dohánění (catch-up)

Hlavní výzvou při návrhu emulátoru je transformace přirozeně paralelního chování hardwarových čipů do sekvenčního kódu hostitelského systému. Na platformách s omezenými zdroji, jako je mikrokontrolér RP2350, se paralelní či událostní modely ukazují jako neefektivní kvůli vysoké režii synchronizace a složitosti implementace. Jako optimální řešení se proto jeví sekvenční metoda dohánění (*catch-up*), která využívá jeden komponent (nejčastěji CPU) jako časovou referenci. Ostatní části systému pak simulují svůj stav dávkově až ve chvíli, kdy potřebují „dohnat“ počet cyklů vykonaných tímto opěrným prvkem (obr. 1.1), což zajišťuje vysoký výkon při zachování nezbytné věrnosti simulace.



Obrázek 1.1: Znázornění catch-up mechanismu na cyklové ose.

Kód 1.1 demonstruje příklad těla smyčky sekvenčního emulátoru, emulujícího abstraktní systém, který se skládá z CPU a n komponentů. Na začátku cyklu emulátoru se určí počet cyklů, které má dosáhnout CPU (`cycles_to_execute`). Když CPU dokončí vykonávání, následně všechny komponenty musí dohnat stav odpovídající současnému stavu CPU. Funkce `Sync()` zajišťuje syn-

chronizovaný a stabilní běh emulátoru (např. podle obnovovací frekvence monitoru).

Počet cyklů, které musí dosáhnout CPU, může být libovolný, ale mohou existovat určité faktory, podle kterých se dá určit optimální hodnota. Např. u systému, kde určitý komponent může vygenerovat přerušení, CPU by měl zastavit těsně před nebo přesně v tom cyklu, kdy má dojít k registraci přerušení procesorem.

while not quit then

$cycles_to_execute \leftarrow cpu.cycles + \dots$

while $cpu.cycles < cycles_to_execute$ do

CpuStep()

end while

C1RunUntil($cpu.cycles * C1_CYCLES$)

C2RunUntil($cpu.cycles * C2_CYCLES$)

...

CnRunUntil($cpu.cycles * Cn_CYCLES$)

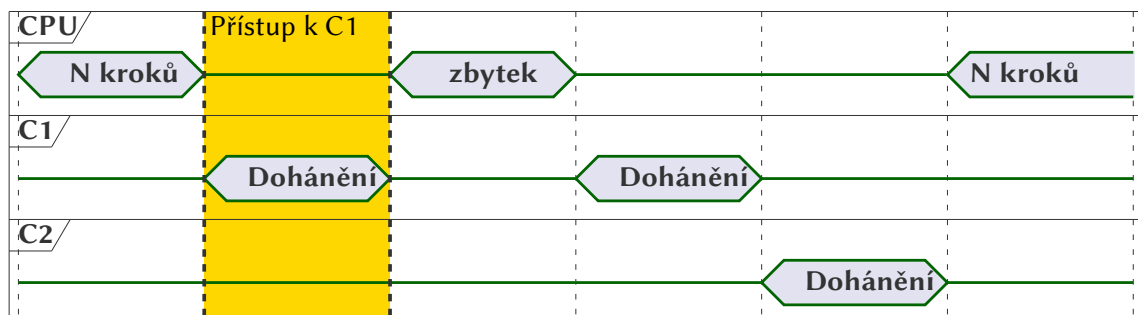
Sync()

end while

Kód 1.1: Konceptuální realizace metody dohánění (*catch-up*) pro více-komponentní systém.

Důležitým je, aby žádný z komponentů nepředběhl CPU, a ani nezaostal, jinak to může způsobit desynchronizaci celého systému nebo jeho části. Synchronizačním bodem musí být striktně současný bod CPU na cyklové ose.

Funkce CpuStep() vykoná jednu instrukci, aktualizuje stav CPU a posune ho po časové ose. Každý n -tý komponent má funkci CnRunUntil(), která aktualizuje stav tohoto komponentu a také ho posune po cyklové ose.



Obrázek 1.2: Znázornění dohánění na požadavek.

Emulované komponenty také mají dohánět, když CPU k nim přistupuje, a přitom CPU ještě nevykonal potřebný počet cyklů. Toto rozšíření catch-up mechanismu se nazývá *catch-up on demand* – dohánění na požadavek (obr. 1.2).

```

procedure CPUSTEP
    while cycles < target_cycles do
        opcode ← Fetch()
        ▷ Způsob dekódování instrukcí závisí na emulované architektuře.
        op_addr ← Decode(opcode)
        cycles ← cycles + Exec(opcode, op_addr)
    end while
end procedure
procedure FETCH
    opcode ← memory[PC]
    PC ← PC + 1
    return opcode
end procedure
procedure EXEC(opcode, op_addr)
    ▷ Každá instrukce má přiřazenou implementační funkci.
    instr ← instr_table[opcode]
    cycles ← instr(op_addr)
    return cycles
end procedure

```

Kód 1.2: Emulace CPU pomocí interpretu.

1.4 Emulace CPU

Při emulaci CPU hlavním zájmem je emulace výsledku instrukcí a vnějšího chování procesoru. Komplexnější procesory, které mohou obsahovat složitý pipeline, připravovat instrukce napřed apod. obvykle se neemuluje.

Nejjednodušší a nejrozšířenější způsob emulace CPU je **interpret**. Spočívá v přímé emulaci instrukčního cyklu (*fetch-decode-execute*) Interpret čte instrukci z paměti a následně ji dekóduje – určuje adresovací režimy, případně další informace, a vypočítává adresu kde je uchován operand. Pak je instrukce přímo vykonána (kód 1.2).[2]

Nevýhodou této metody je relativně nižší výkon. Každá instrukce musí být nejprve načtená z paměti, následně dekódována a až poté vykonána odpovídající obslužnou rutinou. Tento proces vyžaduje několik operací hostitelského procesoru pro vykonání jediné instrukce emulovaného systému. Ve výsledku emulace může být řádově pomalejší než nativní běh programu na původní architektuře.

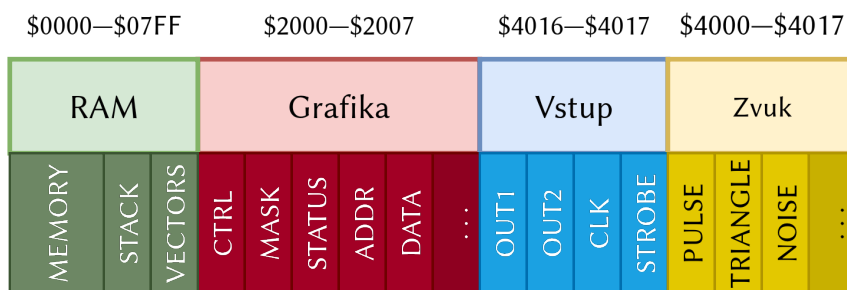
Výhodou je jednoduchost implementace (v jazycích odvozených od C obvykle stačí jazyková konstrukce switch nebo tabulka ukazatelů na funkce). Navíc výsledný kód je snadno přenositelný mezi různými hostitelskými systémy.

Alternativní cestou emulace CPU je dynamická rekompile (JIT – *Just-In-Time compilation*), což je způsob, když emulátor překládá bloky instrukcí emulovaného procesoru do nativního kódu hostitelské architektury. Tento přístup může výrazně zvýšit výkon emulace, ovšem je mnohem složitější v implementaci, a u relativně jednoduchých architektur jako 6502 se nevyplatí, proto

nebude rozebírán v této práci.

1.5 Emulace pamětí a sběrnice

Sběrnice a paměť tvoří komunikační páteř systému, která v kontextu emulace nepředstavuje fyzické vodiče, ale logickou vrstvu řídící veškerý tok informací. Zatímco u nejjednodušších architektur lze paměť reprezentovat jako prosté lineární pole přístupné přímým indexováním, u komplexnějších systémů, poskytujících pokročilé funkce nebo přístup k perifériím, není adresní prostor homogenní, jelikož se zde aplikuje paměťové mapování – technika, při níž jsou konkrétní adresní rozsahy pevně přiřazeny různým hardwarovým entitám, jako jsou moduly RAM a ROM nebo specifické registry periférií.



Obrázek 1.3: Příklad mapované paměti: adresový prostor CPU systému NES.

Emulovaná sběrnice tak funguje jako dispečer, který zachytává každý požadavek procesoru na čtení či zápis a na základě definované paměťové mapy jej přesměruje k příslušnému datovému zdroji nebo k obslužné funkci konkrétního hardwaru. Vzhledem k tomu, že k paměti se přistupuje prakticky v každém hodinovém cyklu (zejména u Von Neumannových architektur), je rychlost tohoto adresového dekódování kritická pro celkový výkon systému. V praxi se využívají dva základní přístupy:

1. **Rozvětvené rozhodování** – emulátor testuje adresu proti pevným rozsahům. Tento přístup je sice programátorsky jednoduchý, ale při velkém množství periferních zařízení vede k vysoké režii z důvodu mnoha logických porovnání.
2. **Stránkování a tabulka ukazatelů** – paměťový prostor je rozdělen na bloky fixní velikosti (např. 1 kB nebo 4 kB). Emulátor udržuje tabulku (pole) funkčních ukazatelů nebo adres, kde pro každou „stránku“ adresního prostoru existuje přímý odkaz na obslužnou rutinu nebo segment paměti. K ní pak přistupuje pomocí bitového posunu $page_idx = addr \gg shift$. Tato metoda je výrazně flexibilnější a umožňuje rychlé přesměrování požadavků bez složitých podmínek.

1.5.1 Přepínání bank

Staré systémy využívaly techniky přepínání bank k tomu, aby programy mohly adresovat více pamětí, než to dovolovává šířka adresní sběrnice. Fyzická paměť byla rozdělena menší bloky, z nichž je v daný okamžik do adresového prostoru CPU zpřístupněná pouze část.

Emulátor v takovém případě uchovává veškerá data v rozsáhlých polích (např. v paměti hostitelského systému), ale procesoru zpřístupňuje pouze aktivní „okna“. Při přepnutí banky programem nedochází k fyzickému kopírování dat, ale pouze ke změně ukazatele v tabulce stránek emulované sběrnice na jinou část paměti.

Příkladem mohou sloužit herní kazety systému NES. Typicky tyto kazety obsahovaly více paměti, než kolik byl CPU schopen přímo adresovat, a proto využívaly speciální obvody umožňující přepínání paměťových bank za běhu programu. Díky tomu hry mohly být mnohem rozsáhlejší. Podrobněji o tom bude popsáno v podkapitole 2.1.

1.5.2 I/O a vedlejší efekty (Side Effects)

Většina moderních a historických systémů využívá paměťově mapované I/O, kde periferie komunikují s CPU prostřednictvím specifických adres v paměťovém prostoru. Emulace sběrnice zde funguje jako most.

Klíčovým konceptem jsou tzv. vedlejší efekty při přístupu k paměti:

- V případě čtení – přístup na adresu registru periferie může v emulátoru způsobit změnu stavu systému (např. vymazání příznaku přerušení nebo posun interního čítače). Pouhý náhled na hodnotu tedy mění logický stav stroje.
- V případě zápisu – zápis na specifickou adresu neukládá hodnotu do paměti, ale přímo ovlivňuje chování hardwaru (např. změnu barvy pixelu nebo spuštění DMA přenosu).

Pro každou takovou adresu musí být v emulátoru realizovány funkce čtení a zápisu.

1.6 Dlaždicová grafika

Grafická emulace je obvykle nejnáročnější částí vývoje, neboť vyžaduje přesnou synchronizaci s procesorem a spotřebovává většinu výpočetního výkonu hostitelského systému. Historické systémy (8bitové a 16bitové) nepracovaly s moderními GPU API, pracujícími s geometrií a vertexy, ale spoléhaly na specializované čipy označované jako **VDP** (*Video Display Processor*) nebo **PPU** (*Picture Processing Unit*). Tyto jednotky přebíraly zátěž spojenou s vykreslováním a umožňovaly obejít tehdejší limity kapacity paměti RAM a výkonu CPU.

V rámci této práce bude popsána **dlaždicová grafika** (*tile-based graphics*), která je nejrozšířenější technika tvorby 2D grafiky v historických 8bitových i 16bitových architekturách.



Obrázek 1.4: Ukázka dlaždicové grafiky v systému NES: zatímco mřížka pozadí je fixní, mřížka jednotlivých spritů se může volně pohybovat. Pořízeno z emulátoru Mesen.

Pro úsporu paměti se obraz neskládá z jednotlivých pixelů, ale z mřížky opakovaně použitelných bloků – dlaždic (*tile*) o velikosti typicky 8×8 pixelů. Přitom pixely těchto dlaždic nejsou skutečné barvy, ale indexy do palet. Vzory dlaždic jsou uchovány v tabulkách vzorů (*Pattern Table* nebo *Tile Set*). Pro rozmísťování dlaždic na obrazovce, systém obvykle udržuje mřížku odkazů na tyto vzory (*Nametable* nebo *Tile Map*). Tento přístup umožňuje efektivní vykreslování rozsáhlých scén a plynulý posun obrazu (*scrolling*) pouhou změnou hodnot v hardwarových registrech (systémy jako NES dokonce podporují scrolling s přesností na jeden pixel).

Pro vykreslování pohyblivých dlaždic, které mohou překrývat dlaždice pozadí, se používají tzv. **sprity** (nebo také objekty). Sprite představuje dvourozměrnou bitmapu, která není vázaná na mřížku dlaždic a může obsahovat průhledné pixely. V systémech jako NES, SNES a Game Boy na sprite je vyhrazená speciální paměť, která se nazývá **OAM** (*object attribute memory*).

1.6.1 Softwarové přístupy k vykreslování v emulátoru

Zatímco původní hardware generoval obraz postupně, v synchronizaci s paprskem CRT monitoru (případně obnovováním LCD u kapesních konzolí), emulátor běžící na moderním systému musí tento proces simulovat. V praxi se volí mezi dvěma hlavními přístupy:

- **Snímkové vykreslování (Frame-based rendering)** – emulátor nechá běžet CPU po dobu, která odpovídá hardwarovému vykreslení celého snímku, a následně celý obraz vygeneruje najednou. Tato metoda je výpočetně nejméně náročná, ale selhává u her, které mění grafické registry mezi řádky.
- **Řádkové vykreslování (Scanline-based rendering)** – emulace procesoru se střídá s emulací grafiky po každém vykresleném horizontálním řádku (scanline). Tento přístup více odpovídá chování skutečného hardwaru a poskytuje vyšší přesnost než snímkové vykres-

lování, avšak nepodporuje meziřádkové modifikace (*mid-scanline modifications* neboli také tzv. *rastrové efekty*).

- **Dávkové vykreslování (Batch rendering)** – tento přístup úzce souvisí s catch-up metodou: CPU běží plynule dopředu (nebo po N cyklech) bez neustálé synchronizace s PPU. Pokud však CPU provede zápis do grafického registru (nebo naopak potřebuje přečíst stav grafického čipu), emulátor pozastaví CPU a nechá PPU „dohnat“ ztracený čas (tzv. *catch-up mechanismus*). Grafický čip tak dávkově vygeneruje chybějící pixely či řádky přesně do okamžiku, kdy k interakci došlo.
- **Vykreslování na úrovni pixelů (Pixel-based / Cycle-accurate rendering)** – představuje nejvyšší možnou úroveň emulace. PPU a CPU jsou striktně synchronizovány na úrovni jednotlivých hodinových taktů (či dokonce sub-cyklů). Obraz je generován pixel po pixelu, což umožňuje bezchybnou simulaci i těch nejkompexnějších grafických triků (např. změna palety uprostřed vykreslování jednoho řádku). Zásadní nevýhodou je velká výpočetní náročnost, zejména v případě mikrokontroléru RP2350, který je použit v praktické části.

Pro většinu 8bitových systémů postačí dávkové vykreslování (např. NES nebo Game Boy), jelikož představuje zlatou střední mezi vykreslováním na úrovni pixelů a řádkovým vykreslováním, a má střední implementační složitost.

1.7 Emulace zvuku

Zvukový výstup představuje z hlediska emulace zcela odlišnou výzvu než grafika. Zatímco u obrazu lidské oko toleruje drobné odchylky v časování (například zahození jednoho snímku z šedesáti projde často bez povšimnutí), lidský sluch je poměrně citlivý na jakékoliv výpadky, fázové posuny nebo nepravidelnosti. Seběmenší chyba v synchronizaci se okamžitě projeví jako nepříjemné praskání nebo zkreslení.

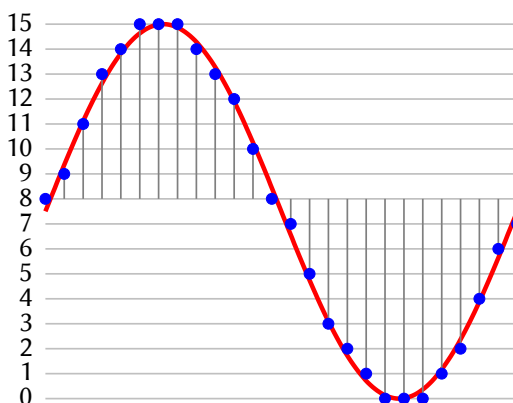
1.7.1 Syntéza zvuku

Většina historických 8bitových a 16bitových (včetně NESu) počítačových systémů nedisponovala pamětí ani výkonem pro přehrávání předem nahraných digitálních vzorků. Místo toho využívaly specializované obvody označované jako **PSG** (*Programmable Sound Generator*). Tyto obvody fungují na principu syntézy zvuku v reálném čase. Obsahují sadu hardwarových oscilátorů, které generují základní vlnové průběhy (čtvercová, trojúhelníková nebo pilovitá vlna) a generátory pseudonáhodného šumu. CPU pouze zapisuje do registrů PSG parametry, jako jsou frekvence (výška tónu), hlasitost a střída (*duty cycle*). Zvukový čip pak na základě těchto parametrů generuje analogový signál.

Kromě PSG, existují také další metody syntézy jako FM, AM. Však v rámci této práce nebudou popsány, jelikož systém NES obsahoval jenom několik PSG a jeden jednoduchý DPCM kanál.

1.7.2 Vzorkovací (sampled) audio

Nejběžnější metoda pro přehrávání neperiodických signálů (a to nejen v historických systémech) je **PCM** (*pulse-code modulation* – pulzně kódová modulace) – signál je v pravidelných časových intervalech vzorkován a amplituda každého vzorku je kvantována na nejbližší hodnotu v rámci definovaného digitálního rozsahu (např. 8 bitů nebo 16 bitů). Přičemž každý vzorek nese absolutní informaci o své hodnotě v daném čase nezávisle na předchozích vzorcích.



Obrázek 1.5: Znárodnění 4bitového PCM: analogový průběh se dělí na diskrétní body. Zdroj [3].

Zajímavým případem je herní konzole NES, která využívala metodu DPCM (*differential pulse-code modulation*) pro přehrávání vzorků, a to její 1bitovou variací. Namísto ukládání absolutní hodnoty každého vzorku se ukládá pouze rozdíl (delta) oproti předchozímu vzorku. Pokud je přečtený bit 1, výstupní úroveň se zvýší. Pokud je bit 0, úroveň se sníží. Zvuková vlna se tak „kreslí“ jako posloupnost malých schůdků nahoru a dolů. Díky tomu bylo možné vměstnat srozumitelný zvuk do zlomku paměti, kterou by vyžadovalo PCM (avšak kvalita zvuku je znatelně horší).

1.7.3 Resampling a aliasing

Historické zvukové čipy byly zpravidla taktovány přímo z hlavního oscilátoru systému, což znamená, že jejich vnitřní stav se měnil na frekvencích v řádech megahertzů (např. krok oscilátoru proběhl téměř dva milionkrát za sekundu). Moderní audio zařízení však vyžadují data v mnohem nižších a standardizovaných vzorkovacích frekvencích, typicky 44,1 kHz nebo 48 kHz. V tomhle případě je potřeba přiběhnout k **downsamplingu**, tedy snížení datového toku při zachování srozumitelnosti (s minimální ztrátou kvality) zvukového výstupu. Jednou z populárních metod downsamplingu je **decimace**, která spočívá ve filtraci signálu pomocí dolní propustí (low-pass filter) a následném výběru N -tého vzorku.[4]

Kapitola 2

Architektura NES

Tato kapitola popíše vnitřní architekturu herního systému NES, kterou následně budeme emulovat v praktické části. Primárním zdrojem informací slouží [5] a [6], které detailně popisují celý systém NES. V následujících podkapitolách budou představeny nejpodstatnější informace, a to pro NTSC verzi, jelikož ji používal americký NES a japonský Famicom (původní NES).

2.1 Herní kazety a správa paměti (Mappers)

Na rozdíl od moderních konzolí, kde médium (CD, DVD, SD karta) slouží pouze jako pasivní úložiště dat, byla herní kazeta pro NES (*cartridge*) přímým rozšířením hardwaru konzole. Po zasunutí do slotu, čipy na kazetě se mapovaly do adresního prostoru CPU i PPU. Existují stovky zaregistrovaných mapperů, ale nejvyužívanější jsou NROM, MMC1, UNROM a MMC3.

2.1.1 Struktura herní kazety a ROM obraz

Každá standardní hra se skládá ze dvou základních typů paměti:

- **PRG-ROM (Program ROM)** – obsahuje kód hry a data pro procesor CPU. V adresním prostoru CPU je pro tuto paměť vyhrazen rozsah od 0x8000 do 0xFFFF (celkem 32 kB).
- **CHR-ROM (Character ROM)** – obsahuje grafická data (dlaždice) pro procesor PPU. Tato data jsou mapována přímo do adresního prostoru grafického čipu. Pokud kazeta neobsahuje CHR-ROM, využívá tzv. **CHR-RAM**, do které si CPU nahraje grafiku z PRG-ROM při startu hry.

Digitální soubor, se kterým emulátor pracuje, se nazývá **ROM obraz** (nebo zkráceně „ROM“). Ten vzniká procesem zvaným *dumping*, kdy je obsah těchto fyzických čipů bit po bitu vyčten a uložen do jednoho souboru. Aby však emulátor věděl, jak má s těmito daty naložit, musí být doplněna metadata o hardwaru původní kazety.

2.1.2 Formát iNES

Standardem pro emulaci NES se stal formát .nes, vycházející ze specifikace iNES. Tento formát navrhl Marat Fayzullin v roce 1996 pro svůj emulátor iNES a dodnes jde o nejrozšířenější způsob distribuce NES her. Klíčovým prvkem je 16bajtová hlavička na začátku souboru (tabulka 2.1), která popisuje konfiguraci hardwaru.

Byte	Význam	Popis
0–3	Identifikátor	ASCII řetězec „NES“ následovaný hodnotou 0x1A.
4	Velikost PRG-ROM	Počet 16 KiB bloků programové paměti.
5	Velikost CHR-ROM	Počet 8 KiB bloků grafické paměti.
6	Příznaky (Flags 6)	Obsahuje typ zrcadlení a spodní 4 bity čísla mapperu.
7	Příznaky (Flags 7)	Obsahuje horní 4 bity čísla mapperu a typ konzole.
8–15	Rezervováno	Většinou nevyužito (nebo použito pro novější standard NES 2.0).

Tabulka 2.1: Struktura hlavičky formátu iNES.

2.1.3 Mechanismy bankování paměti (Mappers)

Hlavním omezením procesoru R2A03 je jeho 16bitová adresní sběrnice, která mu umožňuje přímo adresovat pouze 64 KiB paměti. Z tohoto prostoru je navíc pro hru vyhrazeno pouze 32 KiB, zbytek zabírá RAM, registry periférií, zásobník a vektory přerušení. Jakmile hry začaly svou komplexností tento limit přesahovat, byl na kazety přidáván hardware zvaný **mapper** (oficiálně *Memory Management Controllers* – MMC).

Nejjednodušší mapper implementuje techniku přepínání bank (*bank switching*). Fixní adresní prostor CPU je dynamicky mapován na různé části mnohem větší fyzické paměti ROM na kazetě. Kromě samotného mapování však pokročilé mappery nabízely i další funkce, jako např. vlastní zvukové generátory navíc, PRG RAM (statická RAM paměť místo standardní ROM paměti) nebo vlastní přerušení.

2.2 CPU

Centrální procesorová jednotka (CPU) systému NES je implementována prostřednictvím čipu Ricoh R2A03. Jedná se o 8bitový mikroprocesor odvozený z architektury MOS Technology 6502, od níž se odlišuje primárně absencí podpory binárně zakódované dekadické soustavy (BCD režimu) z důvodu obcházení patentových práv. Čip R2A03 se vyznačuje tím, že na jednom křemíkovém plátu integruje jak samotné jádro procesoru, tak i obvody zvukového generátoru (APU, viz podkapitola 2.4), čímž de facto představuje ranou formu systému na čipu (SoC).

Pracovní frekvence procesoru je exaktně odvozena dělením hlavního hodinového signálu (master clock). V systému NTSC činí hlavní hodinový kmitočet přibližně 21,477 MHz, přičemž procesor pracuje s dělitelem 12 na frekvenci 1,789 MHz.

2.2.1 Architektura registrů

Registrová sada procesoru je celkem minimalistická a odpovídá původnímu návrhu architektury 6502. Procesor nedisponuje univerzálními registry obecného použití, nýbrž využívá sadu úzce specializovaných registrů:

- **A (Accumulator)** — 8bitový pracovní registr.
- **X, Y (Index Registers)** — dvojice 8bitových indexových registrů.
- **PC (Program Counter)** — 16bitový čítač instrukcí uchovávající paměťovou adresu aktuálně zpracovávané instrukce.
- **S (Stack Pointer)** — 8bitový ukazatel vrcholu zásobníku. Hardwarový zásobník je fixně alokovan na nulté stránce paměti (přesněji v rozsahu \$0100-\$01FF) a podle konvence roste směrem k nižším adresám od počáteční hodnoty \$01FF.
- **P (Processor Status)** — 8bitový stavový registr, který sdružuje šest příznaků (flags): Carry, Zero, Interrupt Disable, Decimal (v architektuře NES hardwarově deaktivován), Overflow a Negative.

2.2.2 Paměťová organizace a adresování

Procesor adresuje lineární 16bitový prostor o celkové kapacitě 64 KiB. Tento adresní rozsah je sdílen mezi interní RAM pamětí, paměťově mapovanými periferiemi a pamětí, určené pro účely mapovače. Architektura nevyužívá dedikované I/O instrukce; komunikace s periferiemi probíhá standardními instrukcemi pro čtení a zápis do paměti.

Rozsah adres	Velikost (byte)	Alokované zařízení
\$0000-\$07FF	2048	Interní pracovní RAM
\$0800-\$1FFF	6144	Hardwarové zrcadlení interní RAM
\$2000-\$2007	8	Mapované registry grafického koprocesoru (PPU)
\$2008-\$3FFF	8184	Zrcadlení PPU registrů (opakující se bloky 8 bytů)
\$4000-\$4017	24	Registry APU a vstupně-výstupní rozhraní (I/O)
\$4020-\$FFFF	49152	Prostor cartridge (ROM, RAM, mapovací registry)

Tabulka 2.2: Zjednodušená mapa paměťového prostoru CPU

Interní RAM o kapacitě 2 KiB je logicky segmentována. Nultá stránka (Zero Page, \$0000-\$00FF) umožňuje optimalizované adresování pomocí kratších a rychlejších dvoubajtových instrukcí.

Následuje hardwarově vyhrazená stránka zásobníku (\$0100-\$01FF) a zbývající kapacita je ponechána pro aplikační data. V nejvyšších paměťových adresách (\$FFFA-\$FFFF) jsou pevně definovány vektory přerušení, jež systém načítá z připojené cartridge.

2.2.3 Přerušení

Architektura poskytuje následující přerušení:

- **NMI (Non-Maskable Interrupt)** — softwarově nemaskovatelné přerušení, generované primárně grafickým procesorem (PPU) při zahájení vertikálního zatemnění (VBlank). Obslužný vektor tohoto přerušení se načítá z adres \$FFFA-\$FFFB.
- **IRQ (Interrupt Request)** — maskovatelné přerušení. Softwarově jej lze potlačit nastavením příznaku *Interrupt Disable* ve stavovém registru (P). Obvykle slouží pro externí časovače integrované v mapperu cartridge, nebo pro DMC v APU. Obslužný vektor sídlí na adresách \$FFFE-\$FFFF.
- **RESET** — hardwarová inicializace spuštěná při startu konzole nebo po stisku tlačítka Reset. Z hlediska vnitřní logiky CPU funguje jako specifické přerušení s tím rozdílem, že potlačuje instrukce zápisu návratové adresy na zásobník a automaticky nastavuje příznak *Interrupt Disable*. Vektor inicializace se nachází na \$FFFC-\$FFFD.

2.2.4 Adresovací režimy

Instrukční sada procesoru 6502, z níž čip R2A03 vychází, specifikuje celkem třináct adresovacích režimů. Tyto režimy definují způsob, jakým operační kód přistupuje k operandu, což má přímý dopad na délku instrukce (v bytech) a počet strojových taktů nezbytných k jejímu zpracování. Adresovací režimy jsou podrobně popsány v datasheetu [7].

2.2.5 Neoficiální instrukce

Oficiální specifikace architektury 6502 definuje 151 validních instrukčních kódů. Ze zbývajících 105 kombinací v rámci 8bitového prostoru operačních kódů je většina na fyzickém čipu R2A03 reálně spustitelná. V odborné literatuře a scéně emulátorů se tyto instrukce označují jako *neoficiální* či *nezdokumentované instrukce* (unofficial/illegal opcodes).

Existence těchto kódů není zamýšlenou funkcí, nýbrž artefaktem způsobu, jakým čip dekoduje instrukce. Řadič procesoru nevyužívá mikrokód v tradičním slova smyslu, ale kombinační logiku řízenou programovatelným logickým polem (PLA — *Programmable Logic Array*) obsahujícím 130×21 (21 bitů) řádků. Neoficiální operační kódy nečekaně aktivují více řádků PLA současně. Tím dochází ke spuštění nesouvisejících větví dekodéru a čip vykoná operaci slučující chování

Adresní rozsah	Velikost	Účel	Mapování
\$0000–\$0FFF	4096 B	Tabulka vzorů 0 (Pattern Table)	Kazeta
\$1000–\$1FFF	4096 B	Tabulka vzorů 1 (Pattern Table)	Kazeta
\$2000–\$23BF	960 B	Jmenná tabulka 0 (Nametable)	Interní/VRAM
\$23C0–\$23FF	64 B	Tabulka atributů 0	Interní/VRAM
\$2400–\$2FFF	—	Zrcadlení jmenných tabulek 1-3	Dle zapojení
\$3F00–\$3F0F	16 B	Paleta pozadí	Interní
\$3F10–\$3F1F	16 B	Paleta spritů	Interní

Tabulka 2.3: Paměťová mapa PPU.

několika standardních instrukcí.[8] Popis a charakteristika jednotlivých neoficiálních instrukcí je popsána v [9].

2.3 PPU

Obrazový procesor (PPU – *Picture Processing Unit*) je hlavní komponentem systému NES zodpovědným za generování výsledného video signálu na základě dat z procesoru a grafických dat uložených na herní kazetě. PPU generuje kompozitní signál sestávající z 262 řádků (*scanlines*), z nichž je 240 viditelných.

PPU funguje na principu dlaždicové grafiky, kde pozadí je rozděleno na menší bloky (dlaždice) o velikosti 8×8 pixelů. Samotné dlaždice se uchovávají v CHR paměti kazety, která je mapována do adresního prostoru PPU. Také PPU je vybaven 8 paletami.

2.3.1 Paměťová organizace a sběrnice

PPU disponuje vlastní sběrnicí, ke které je připojena interní video paměť (VRAM) a dlaždicová paměť na kazetě (CHR) (tabulka 2.3). Pro potřeby emulace je zásadní rozlišovat mezi vykreslováním pozadí a pohyblivých objektů (spritů), neboť každý z těchto prvků využívá jinou část paměťové hierarchie.

Fyzicky PPU obsahuje pouze 2 KiB paměti pro jmenné tabulky (dostačující pro dvě tabulky), zbývající dvě logické tabulky jsou řešeny zrcadlením (*mirroring*), které může být horizontální nebo vertikální v závislosti na hardwarovém zapojení na kazetě (mapperu).

2.3.2 Vystavené registry

PPU mapuje devět registrů do adresního prostoru CPU:

1. PPUCTRL – základní nastavení činnosti PPU
2. PPUMASK – nastavení procesu vykreslování

3. PPUSTATUS - stavový registr (V-Blank, NMI; jenom pro čtení)
4. PPUSCROLL - nastavení scrollingu
5. PPUADDR - adresa do VRAM
6. PPUDATA - data do VRAM
7. OAMADDR - adresa do primárního OAM
8. OAMDATA - data do primárního OAM
9. OAMDMA - DMA kanál do primárního OAM

2.3.3 Vzory dlaždic a tabulky vzorů (Pattern Tables)

Vzory dlaždic jsou uloženy v tabulkách vzorů v paměti CHR na kazetě. Aby se ušetřilo místo, NES využívá metodu bitových rovin (*bitplanes*). Každý řádek dlaždice je definován dvěma byty: první byte obsahuje nižší bity barvy pro 8 pixelů, druhý byte (uložený o 8 bytů dále) obsahuje vyšší bity. Výsledkem je 2bitový index (hodnota 0–3), který určuje barvu pixelu v rámci zvolené palety (obr. 2.1).

PPU je schopen adresovat pouze 2 tabulky vzorů. Existují však mappery, které implementují přepínání banek tabulek vzorů a proto dovolují používat více než 2 tabulky.

0	0	0	0	0	3	3	3	3	0	0	0	0	0	0	0
0	0	0	0	3	3	3	3	3	3	3	0	0	0	0	0
0	0	0	0	1	1	1	1	2	2	0	0	0	0	0	0
0	0	0	1	2	2	1	2	2	1	2	2	2	0	0	0
0	0	0	1	2	2	1	1	2	2	1	2	2	2	0	0
0	0	1	1	1	2	2	2	2	1	1	1	1	0	0	0
0	0	0	0	0	2	2	2	2	2	2	2	0	0	0	0
0	0	0	0	1	1	1	1	1	1	0	0	0	0	0	0
0	0	0	1	1	1	1	3	3	1	1	0	0	0	0	0
0	0	0	1	1	1	3	3	2	3	3	0	0	0	0	0
0	0	0	1	1	1	1	3	3	3	3	0	0	0	0	0
0	0	0	3	1	2	2	2	3	3	3	3	0	0	0	0
0	0	0	3	3	2	2	3	3	3	3	3	0	0	0	0
0	0	0	3	3	3	3	0	3	3	3	0	0	0	0	0
0	0	0	1	1	1	0	0	1	1	1	0	0	0	0	0
0	0	0	1	1	1	1	0	1	1	1	1	0	0	0	0

(a) Rozvržení spritu s indexy do palety (barva 0 je průhledná).

0	0	0	0	0	3	3	3	3	0	0	0	0	0	0	0
0	0	0	0	3	3	3	3	3	3	3	3	0	0	0	0
0	0	0	0	1	1	1	1	2	2	0	0	0	0	0	0
0	0	0	1	2	2	1	2	2	1	2	2	2	0	0	0
0	0	0	1	2	2	1	1	2	2	1	2	2	2	0	0
0	0	1	1	1	2	2	2	2	1	1	1	1	0	0	0
0	0	0	0	0	2	2	2	2	2	2	2	0	0	0	0
0	0	0	0	1	1	1	1	1	1	0	0	0	0	0	0
0	0	0	1	1	1	1	3	3	1	1	0	0	0	0	0
0	0	0	1	1	1	3	3	2	3	3	0	0	0	0	0
0	0	0	1	1	1	1	3	3	3	3	3	0	0	0	0
0	0	0	3	1	2	2	2	3	3	3	3	0	0	0	0
0	0	0	3	3	2	2	3	3	3	3	3	0	0	0	0
0	0	0	3	3	3	3	0	3	3	3	0	0	0	0	0
0	0	0	1	1	1	0	0	1	1	1	0	0	0	0	0
0	0	0	1	1	1	1	0	1	1	1	1	0	0	0	0

(b) Obarvený sprite (barvy závisí na specifikované paletě v metadatech spritu).

Obrázek 2.1: Indexování do palet pomocí bitových rovin. Zdroj [10].

2.3.4 Jmenné tabulky (Nametables)

Jmenná tabulka definuje rozvržení pozadí obrazovky. Skládá se z 30 řádků po 32 dlaždicích, přičemž každý byte v tabulce představuje index dlaždice z *Pattern Table*.

2.3.5 Tabulky atributů (Attribute Tables) a palety

Vzhledem k limitacím hardwaru nemůže mít každá dlaždice vlastní libovolnou barvu. Tabulka atributů (posledních 64 bajtů v každé jmenné tabulce) rozděluje obrazovku na bloky 4×4 dlaždice (tzv. metatily). Každý bajt v této tabulce určuje paletu pro čtyři sousedící skupiny dlaždic (každá o velikosti 2×2 dlaždice).



Obrázek 2.2: Hardwarová paleta systému NES. Zdroj [11].

PPU využívá celkem 8 palet po 4 barvách (4 pro pozadí a 4 pro sprity). První barva v každé paletě je považována za průhlednou (u pozadí se místo ní vykresluje barva pozadí z paměti \$3F00). Ačkoliv PPU interně pracuje s 53 barvami (obr. 2.2), díky speciálním bitům v registru PPUMASK a přesnému časování (tzv. *color emphasis*) lze dosáhnout širší barevné škály.[12]

2.3.6 Objekty a sprity (OAM)

Data o spritech jsou uložena ve vyhrazené paměti **primární OAM** (*Object Attribute Memory*) o velikosti 256 bajtů, což umožňuje definovat až 64 spritů. Každý sprite se skládá z 4 bajtů: souřadnice Y, index dlaždice, atributy (horizontální a vertikální zrcadlení, index palety, priorita vůči pozadí) a souřadnice X.

2.3.7 Evaluace spritů (prohledávání OAM)

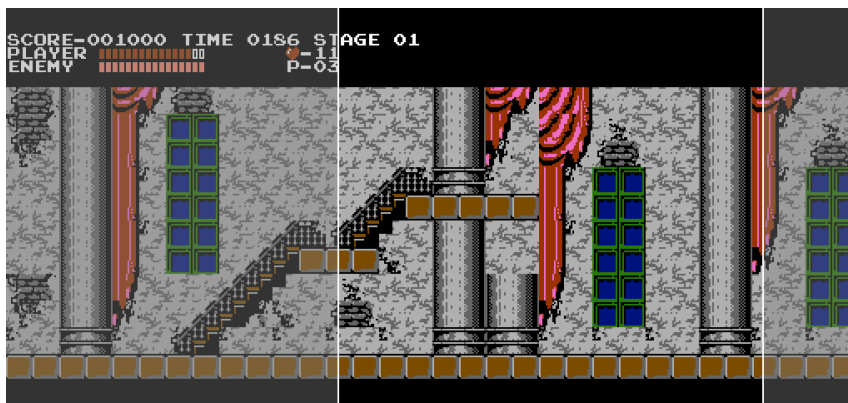
Jednou za scanline, PPU nezávisle na vykreslování prohledává primární OAM paměť pro vykreslení dalších řádků spritů. Vybírá nejvýše 8 spritů a kopíruje jejich data do vnitřní mezipaměti (*sekundární OAM*), které pak využije k vykreslení řádků spritů.

2.3.8 Sprite overflow

V případě, pokud při prohledávání primární OAM již se zaplnil sekundární OAM, PPU ukončí tento proces a nastaví příznak **sprite overflow**. Z důvodu chyby v původním návrhu hardwaru NES, logika detekce přetečení spritů nebyla správná, proto tento příznak se téměř nepoužívá.

2.3.9 Scrolling

Jednou z klíčových vlastností grafického procesoru PPU je podpora scrollování (plynulého posunu) pozadí na úrovni jednotlivých pixelů, a to v horizontálním i vertikálním směru. Tento mechanismus umožňuje definovat výřez (*viewport*) do paměťového prostoru pozadí, který je větší než samotná viditelná plocha obrazovky. Díky tomu nejsou hry omezeny na jedinou statickou scénu, ale mohou vytvářet rozsáhlé herní světy přesahující rozlišení monitoru.



Obrázek 2.3: Znáznornění scrolingu: šedé obdélníky vyznačují viditelnou část jmenných tabulek, zatímco oblast mezi nimi není hráčem viditelná. Hry předem zapisují do neviditelných oblastí jmenných tabulek další dlaždice mapy. Je zde také patrné zrcadlení jmenných tabulek. Pořízeno z emulátoru Mesen.

Ačkoliv každá jmenná tabulka popisuje fixní mřížku dlaždic (32×30), scrollování není omezeno hranicemi jedné konkrétní tabulky. Zásadní roli zde hraje zrcadlení, které definuje mapování fyzických jmenných tabulek v adresním prostoru paměti VRAM. V okamžiku, kdy výřez obrazu překročí hranici aktuální tabulky, zrcadlení určuje, která data budou interpretována jako navazující. Zrcadlení tak efektivně určuje preferovaný směr (horizontální či vertikální), ve kterém může probíhat plynulý posun obrazu bez vzniku grafických artefaktů (obr. 2.3).

Pozice výřezu je definována kombinací souřadnic dlaždic a jemných pixelových offsetů v registrech PPUSCROLL a PPUADDR.

Přesná hardwarová implementace scrollování v původním čipu PPU nebyla po dlouhou dobu veřejně známa. V komunitě vývojářů emulátorů se však stal standardem tzv. **Loopyho model**¹. Tento model popisuje vnitřní registry PPU a jejich specifické chování během vykreslovacího cyklu, což umožňuje dosáhnout vysoké věrnosti emulace i u technicky náročných her (zápis do registrů scrollu během vykreslování, diagonální směr scrolingu apod.).

¹Autorem je uživatel fóra NESDev s přezdívkou „Loopy“, který model odvodil na základě reverzního inženýrství a pozorování reálného hardwaru.

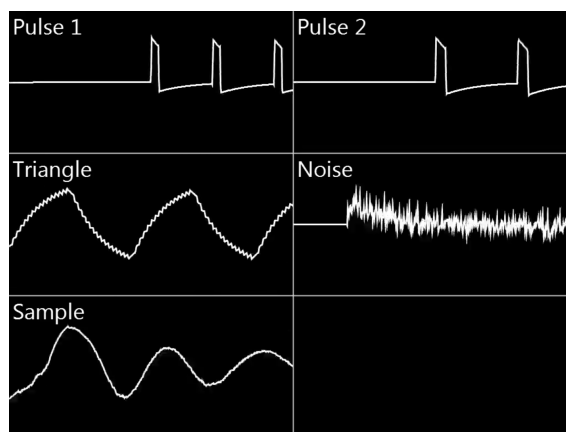
2.3.10 Proces vykreslování a časování

Vykreslování probíhá cyklus po cyklu v rámci každého řádku. Z hlediska emulace je důležité rozlišení fází:

1. **Pre-render scanline (-1 nebo 261):** PPU se připravuje na snímek, plní vnitřní registry a čistí příznaky V-Blank a Sprite 0 Hit.
2. **Visible scanlines (0–239):** Probíhá samotné generování obrazu. V této fázi PPU neustále přistupuje k paměti, proto by CPU nemělo do VRAM zapisovat (hrozí grafické artefakty, ačkoliv velké množství programů to ignoruje).
3. **Post-render scanline (240):** Neaktivní řádek.
4. **V-Blank (241–260):** Obraz se nevykresluje. Na začátku této fáze se nastaví příznak V-Blank a generuje se přerušení NMI. Toto je bezpečné okno pro CPU k aktualizaci grafických dat a přenosu dat do OAM.

2.4 APU

Zvukový procesor APU (*Audio Processing Unit*) není samostatným čipem, ale je integrován přímo do CPU. Jeho registry jsou mapovány do adresového prostoru CPU v rozsahu \$4000–\$4013, \$4015 a \$4017; naprostá většina z nich je pouze pro zápis. APU disponuje pěti zvukovými kanály: dvěma generátory pulzního signálu (Pulse 1, Pulse 2), generátorem trojúhelníkové vlny (Triangle), generátorem šumu (Noise) a kanálem delta modulace pro přehrávání vzorků (DMC).



Obrázek 2.4: Přehled zvukových kanálů APU. Zdroj [6].

2.4.1 Časování

Taktová frekvence APU je přibližně stejná jako u CPU. Většina zvukových kanálů však jsou taktovány poloviční frekvencí nebo i menší; jejich komponenty jsou místo toho řízeny hierarchií

děličů (**dividerů**) a **timerů**, které z tohoto základního taktu odvozují potřebné frekvence.

Klíčovým pojmem je **perioda** (*period*) – udává, kolik APU cyklů musí uplynout, než daná komponenta provede svůj krok. Konkrétní implementační mechanismus se nazývá divider – sestupný čítač, který se dekrementuje každý APU cyklus. Jakmile dosáhne nuly, vyvolá příslušnou akci a je znovu nastaven na hodnotu periody:

$$\text{divider} = \text{period} \implies \text{čekej } \textit{period} \text{ cyklů, pak proved' akci}$$

V hardwaru NES se divider realizuje dvěma způsoby. Pulzní a trojúhelníkový kanál používají lineární sestupné čítače. Kanál šumu, DMC a snímkový čítač jsou implementovány prostřednictvím lineárních zpětnovazebních posuvných registrů (LFSR – *Linear Feedback Shift Register*).

Timer je terminologicky totožný s dividerem – jedná se o jiný název pro stejný mechanismus. V kontextu APU se slovem *timer* zpravidla označuje divider řídící výšku tónu (pitch) kanálu, zatímco ostatní dividery (pro obálku, sweep, délkový čítač) si ponechávají vlastní pojmenování. Toto rozlišení je čistě konvenční.

2.4.2 Struktura kanálů

Každý zvukový kanál se skládá z kombinace následujících funkčních podjednotek; konkrétní sada závisí na typu kanálu:

- **Timer** – divider odvozující výstupní frekvenci kanálu. Hodnota jeho periody je zapisována CPU do příslušných registrů a přímo určuje výšku tónu. Kanál je hardwarově ztišen, pokud perioda klesne pod minimální prahovou hodnotu.
- **Sekvencer** – komponenta procházející pevně danou sekvencí hodnot a předávající aktuální krok na výstup kanálu. Je řízen timerem; každým jeho tickem přejde na další pozici v sekvenci. Sekvencer definuje tvar výstupní vlny – například 8krokovou sekvenci střidy u pulzního kanálu nebo 32krokový trojúhelníkový průběh.
- **Envelope** – generátor obálky řídící okamžitou hlasitost kanálu. Interně jde o divider taktovaný snímkovacím čítačem; po každém svém ticku dekrementuje 4bitový čítač hlasitosti nebo jej (v režimu smyčky) restartuje od maxima. Výsledná hlasitost může být odvozena z tohoto čítače, nebo nastavena konstantně.
- **Sweep** – jednotka automatického přeladování dostupná pouze u pulzních kanálů. Je to divider řízený snímkovacím čítačem, který periodicky upravuje hodnotu timeru (a tedy výšku tónu) o přírůstek daný bitovým posunem. Při překročení rozsahu je kanál automaticky ztišen.
- **Length Counter** – čítač délky tónu přítomný ve všech kanálech kromě DMC. Umožňuje automatické ztišení kanálu po uplynutí přednastavené doby bez zásahu CPU. Hodnota

délky se načítá z pevné vyhledávací tabulky a je taktována snímkovacím čítačem.

- **Linear Counter** – dodatečný čítač délky specifický pro trojúhelníkový kanál. Oproti length counteru nabízí jemnější rozlišení (7bitová hodnota zadávaná přímo) a je taktován dvakrát tak rychle.

2.4.3 Snímkovací čítač (Frame Counter)

Snímkovací čítač (*frame sequencer*) je centrální řídicí jednotka APU taktující všechny nízko-frekvenční komponenty – envelope generátory, sweep jednotky a délkové čítače. Navzdory svému názvu nemá přímou vazbu na obrazový signál PPU; pojmenování odráží přibližnou shodu jeho výstupní frekvence s obnovovací frekvencí obrazu.

Sekvencer je sám vícekrokový divider: při každém svém ticku aktivuje různé podmnožiny komponent. Konfiguruje se zápisem do registru \$4017 do dvou režimů:

- **4krokový režim** – envelope a linear counter jsou taktovány každý krok (přibližně 240 Hz pro NTSC), sweep jednotky každý druhý krok (přibližně 120 Hz). Na konci čtvrtého kroku může volitelně generovat přerušení IRQ (*frame IRQ*).
- **5krokový režim** – přidáním pátého kroku se celkové tempo modulátorů snižuje (efektivní frekvence klesá na přibližně 48 Hz pro NTSC). Přerušení IRQ v tomto režimu generováno není.

2.4.4 Mixér a výstup

Výstupy všech pěti kanálů jsou kombinovány interním mixérem se dvěma větvemi: pulzní větev (Pulse 1 + Pulse 2) a TND větev (Triangle + Noise + DMC). Každá větev má vlastní nelineární DAC, takže výsledná amplituda není prostým součtem hodnot jednotlivých kanálů. Tato nelinearita způsobuje vzájemné ovlivňování kanálů — například zápis do výstupního registru DMC (\$4011) mění zdánlivou hlasitost trojúhelníkového a šumového kanálu. Pro emulaci mixéru je možné použít vyhledávací tabulku aproximovaných hodnot[13]:

$$\begin{aligned} \text{output} &= \text{pulse}_{\text{out}} + \text{tnd}_{\text{out}} \\ \text{pulse}_{\text{lut}}[n] &= \frac{95,52}{\frac{8128}{n} + 100} \\ \text{pulse}_{\text{out}} &= \text{pulse}_{\text{lut}}[\text{pulse}_1 + \text{pulse}_2] \\ \text{tnd}_{\text{lut}}[n] &= \frac{163,67}{\frac{24329}{n+100}} \\ \text{tnd}_{\text{out}} &= \text{tnd}_{\text{lut}}[3 \cdot \text{triangle} + 2 \cdot \text{noise} + \text{dmc}] \end{aligned}$$

Kapitola 3

Specifikace požadavků

Před zahájením samotného návrhu hardwaru a implementace emulátoru bylo nezbytné definovat sadu kritérií, která zajistí plynulý chod emulace a autentický uživatelský zážitek. Požadavky lze rozdělit na systémové nároky a fyzická rozhraní zařízení.

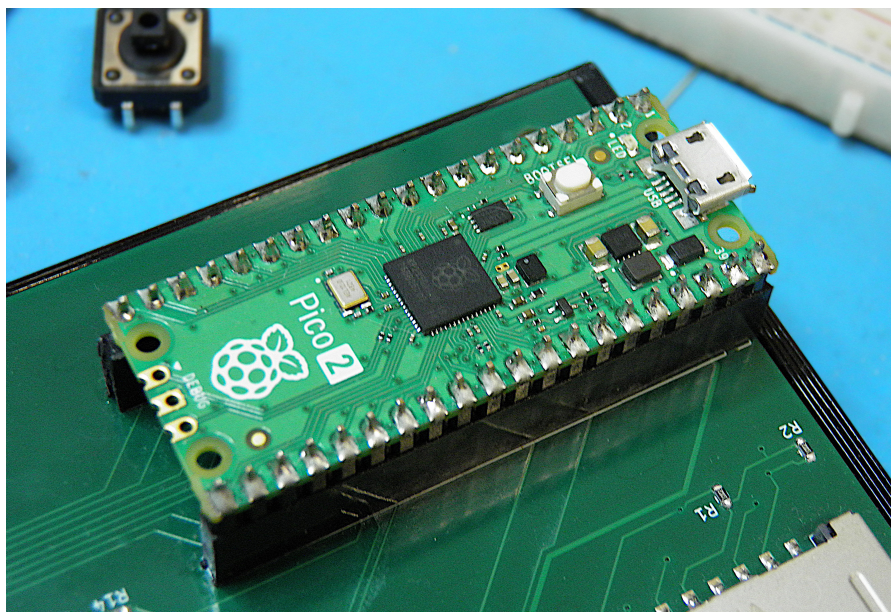
3.1 Software

- NES využíval NTSC jako standard pro analogové televizní vysílání, který pracoval se snímkovým kmitočtem 60 Hz. Emulátor musí generovat obraz stabilně při 60 snímcích za sekundu, aby nedocházelo k trhání obrazu.
- Emulátor musí v reálném čase emulovat CPU, PPU a APU tak, aby byla zachována přesná časová souslednost mezi herní logikou, obrazem a zvukem.
- Z předchozích dvou bodů vychází, že mikrokontrolér musí být dostatečně výkonný aby stíhal emulovat, generovat obraz, zvuk a zpracovávat uživatelský vstup.
- Výsledný systém by měl obsahovat systémové menu, které umožní uživateli spouštět programy a konfigurovat běh celého systému.
- Aby se zachoval kód emulátoru udržitelným a čitelným, systémové menu má být realizováno jako samostatný program, který bude schopen komunikovat s emulátorem, aby spouštět ostatní programy nebo konfigurovat emulátor. To je velice podobný koncepci BIOSu, což je program, určený pro načtení operačního systému a nízkoúrovňovou konfiguraci zařízení.
- Mikrokontrolér musí disponovat dostatkem paměti RAM pro uložení stavu emulátoru (stav jednotlivých komponentů, paměť programů, videopaměť) a dostatek pamětí (např. Flash) pro uložení samotného kódu firmwaru a obrazu BIOSu.
- Emulátor má být napsan v jazyce C bez přímých závislostí na specifickém hardwaru, což usnadňuje ladění na PC a případný budoucí port na jiné architektury.

3.2 Hardware

- Vzhledem k ústupu analogových technologií bude využit digitální standard HDMI, který je kompatibilní s většinou moderních monitorů a televizorů.
- Pro zajištění kvalitního poslechu bez nutnosti implementace komplexního digitálního audio přes HDMI bude zařízení vybaveno standardním 3,5mm Jack konektorem.
- Zařízení musí obsahovat dva porty pro připojení herních ovladačů pro podporu her více hráčů, podobně jako v originálním systému NES. Latence vstupu musí být minimální.
- Systém bude vybaven hlavním vypínačem (*Power*) a dedikovaným tlačítkem *Reset*, které umožní okamžitý návrat z programu do systémového menu (BIOSu).
- Pro ukládání programů a konfigurací bude využito rozhraní pro SD karty, které nabízí vysokou kapacitu při zachování jednoduché implementace a přenositelnosti dat.
- Napájení bude zajištěno přes standardní Barrel Jack konektor. Zařízení musí dále obsahovat USB rozhraní pro snadnou aktualizaci firmwaru uživatelem bez potřeby externího programátoru.

Na základě těchto požadavků byla zvolena vývojová deska **Raspberry Pi Pico 2**, řízená mikrořadičem **RP2350**. Jedním z hlavních důvodů výběru této desky je její poměrně nízká cena (v okamžik psaní této práce, průměrná cena této desky je 120 Kč), a mocný mikrořadič RP2350, parametry kterého jsou popsány v datasheetu [14]. Popis použitých periférií RP2350 je možné najít v příloze G.



Obrázek 3.1: Vývojová deska Raspberry Pi Pico 2 s mikrořadičem RP2350.

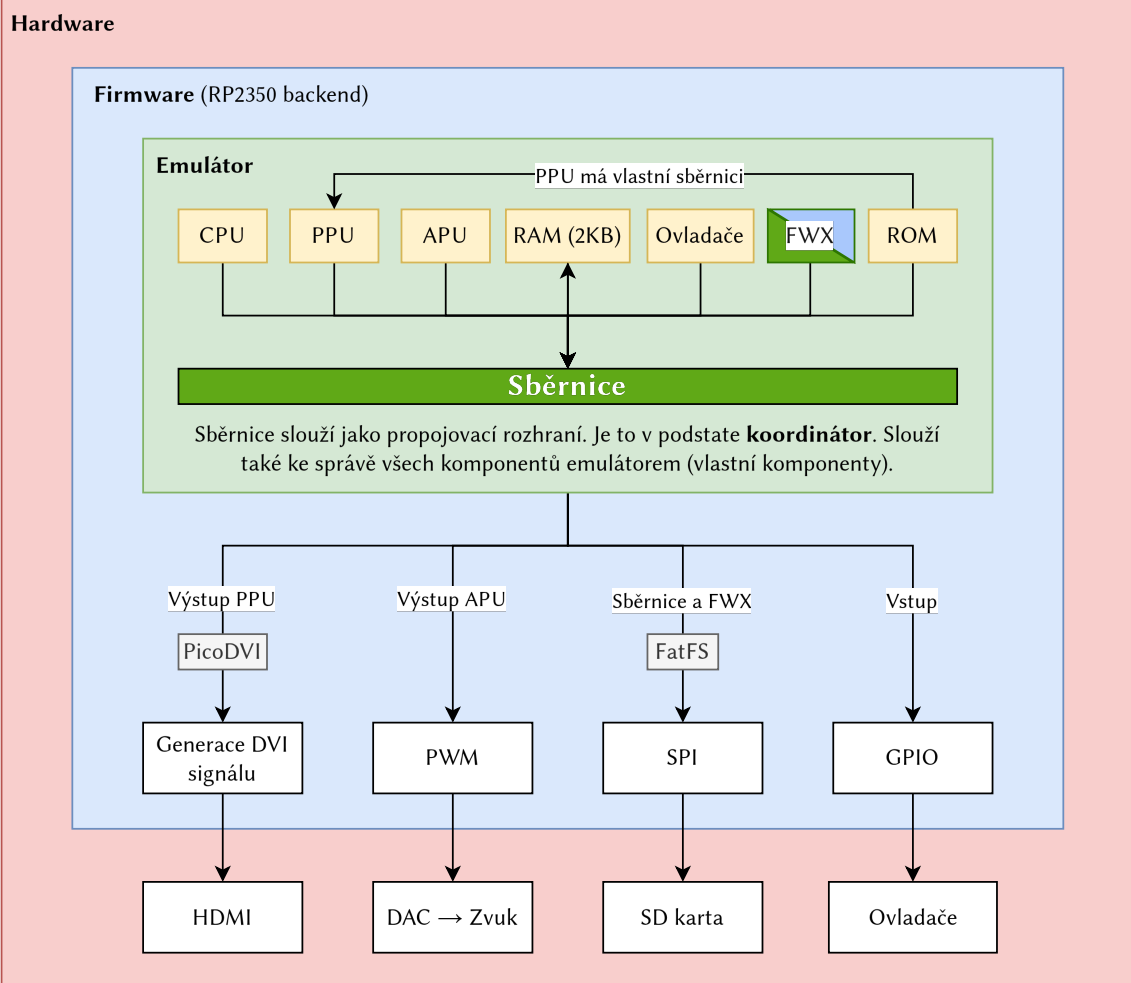
Kapitola 4

Architektura projektu

Cely projekt je rozdělen do tří logických vrstev (viz koncepční schéma 4.1):

- **Emulační vrstva** představuje bezprostředně samotnou emulaci CPU, PPU, APU, sběrnice, ovladačů a ROM obrazu programu. Jádro emulátoru je zcela abstraktní – je nezávislé na platformě, a nemá povědomí o tom, na jakém koncovém zařízení běží. Tato vrstva mimo jiné obsahuje také jeden z hlavních přínosů našeho projektu – komunikační linka FWX, což představuje most mezi emulovaným programem a emulátorem. FWX mapuje do emulované paměti registry, díky kterým BIOS je schopen vyjít za rámec systému NES a komunikovat s emulátorem (například poslat požadavek o načtení programu).
- **Firmware** zajišťuje skutečnou realizaci výstupu a vstupu, a propojuje softwarovou část (emulátor) s hardwarovou částí. Tato vrstva předkládá požadavky emulátoru na reálné události a signály – implementuje výstup obrazu pomocí DVI, generaci audio signálu, zpracovávání vstupu z tlačítek ovladačů, a komunikaci s SD kartou pro načtení programů.
- **Hardware** fyzicky propojuje mikrokontrolér s periferiemi a zajišťuje jeho správnou funkci. Zahrnuje elektrické zapojení mikrokontroléru RP2350, napájení, ošetření signálových cest a integraci veškerých periférií a konektorů (HDMI, audio jack, porty pro ovladače, slot pro SD kartu a napájení).

Jednotlivé komponenty a technická řešení všech tří vrstev jsou detailně rozebrány v následujících kapitolách.



Obrázek 4.1: Konceptní schéma architektury projektu, která je postavena na 3 vrstvách: emulátor, firmware a hardware.

Kapitola 5

Struktura zdrojového kódu

Při vývoji byl kladen důraz na modularitu a čisté oddělení platformově nezávislého jádra emulátoru od specifické implementace pro cílový hardware (video, zvuk a vstup). Tato architektura umožňuje snadné testování a ladění logiky emulace na osobním počítači (desktopový backend) před samotným nasazením na mikrokontrolér.

Zdrojové soubory jsou organizovány do logických celků, které odpovídají funkčním blokům systému NES:

docs/	dokumentace
soc/	zdrojový kód tohoto skriptu
include/	
fwkes/	.h soubory projektu
cpu.h	deklarace CPU
ppu.h	deklarace PPU
...	
src/	.c soubory projektu
bios/	zdrojový kód BIOSu
rp2350/	backend pro RP2350
desktop/	backend pro PC
cpu.c	implementace CPU
ppu.c	implementace PPU
...	
tests/	testovací kód
third-party/	knihovny třetích stran
CMakeLists.txt	automatizace sestavování

Organizace zdrojového kódu projektu.

Jelikož cílovou architekturou je mikrokontrolér RP2350, celý kód a struktura softwaru musí být navržena tak, aby výsledný strojový kód byl dost rychlý a optimalizovaný na RP2350. Také neméně důležité je efektivní správa paměti a její úspora, jelikož interní SRAM paměť je pouhých 520 KB. Jednotlivé techniky optimalizace budou popsány dále v textu.

Kapitola 6

Návrh emulátoru

Emulátor je navržen jako nízkoúrovňový, což znamená, že emuluje na úrovni jednotlivých instrukcí procesoru, registrů a cyklů. Tento přístup zajišťuje lepší kompatibilitu s původním softwarem, který často využíval nedokumentované vlastnosti hardwaru NES.

Vzhledem k tomu, že RP2350 disponuje omezenými prostředky pro paralelní zpracování s vysokou režii (např. multitasking v reálném čase), je emulátor navržen jako sekvenční a využívá catch-up metodu (viz 1.3) pro synchronizaci komponentů. Paralelní běh by vyžadoval náročnou synchronizaci a zamykání prostředků, takže tento přístup zjednodušuje návrh a je optimální na RP2350.

CPU je řídicí prvek: PPU a APU „spí“, dokud CPU nevykoná určitý počet cyklů nebo nepokusí se s nimi komunikovat. V ten moment tyto komponenty dohánějí stav aktuální stav CPU. Díky tomuto modelu je emulátor zároveň cyklově přesný. Každá instrukce CPU trvá přesně stejný počet emulovaných taktů jako na reálném hardwaru, což je kritické pro hry, které využívají přesné časování pro grafické efekty (např. změna palety uprostřed vykreslování řádku, nebo zápis do registru scrollu).

Pro reprezentaci komponentů v kódu se používají struktury, které zapouzdřují celý vnitřní stav komponentů. Struktury neobsahují pouze registry, ale i vnitřní čítače, příznaky a případně buffery. To umožňuje v budoucnu například snadné ukládání stavu systému do souboru a jeho opětovné načtení. Ke každé struktuře náleží sada funkcí, zpravidla zodpovědné za inicializaci, resetování, aktualizaci stavu a sběrníkové funkce (`read()`/`write()`).

6.0.1 Smyčka emulátoru

Smyčka emulátoru provádí emulaci po snímcích PPU (viz výpis 1). Nejdřív se určí počet cyklů, které vykoná CPU, a až pak se aktualizuje stav PPU a APU (dohánějí CPU). Pokud při dohánění na vyžádání PPU dokončí vykreslování snímku, CPU musí ihned ukončit vykonání, aby nedošlo k desynchronizaci.

Potřebný počet cyklů pro CPU se určuje na základě nejbližší významné události – stav VBlank, přerušení NMI nebo konec vykreslovaného řádku. Pro NTSC verzi systému NES platí že za jeden řádek CPU vykoná cca 113,67 cyklů.

```

/* Emulace se provádí po snímcích */
while (!ppu->frame_done) {
    CycleCounter cycles_to_run = /* ... */;
    /* CPU provede určitý počet cyklů */
    while (cpu->cycles < cycles_to_run && !ppu->frame_done) {
        cpu_step(cpu);
    }
    /* Dohánění */
    ppu_run_until(ppu, cpu->cycles * 3);
    apu_run_until(apu, cpu->cycles);
}

ppu->frame_done = false;

```

Výpis 1: Smyčka emulátoru. CPU provádí určitý počet instrukcí, a pak PPU a APU dohánějí jeho stav.

6.1 ROM obrazy a mappery

Abstrakce herní kazety a její vnitřní logiky je realizována skrze strukturu `Disk`. Tato struktura zapouzdřuje veškerá metadata o načteném ROM obrazu, ukazatele na alokované paměťové bloky (PRG a CHR) a aktuální vnitřní stav aktivního mapperu.

Inicializace struktury probíhá pomocí funkcí `disk_load()` (pro načítání ze souborového systému SD karty) a `disk_load_mem()` (pro přímé zavedení z paměti RAM). Obě funkce provádějí parsování a validaci hlavičky formátu iNES, inicializaci mapperu a alokaci PRG a CHR pamětí na základě zjištěných údajů.

Vzhledem k tomu, že každý mapper vyžaduje pro svou činnost odlišné řídicí proměnné – jako jsou čítače přerušení, registry bankování nebo latch stavy – bylo nutné vyřešit jejich ukládání. Pro úsporu paměti je stav mapperu v rámci struktury `Disk` definován jako sjednocení (union). Tento přístup zajišťuje, že v paměti je vyhrazen pouze takový prostor, který odpovídá nejnáročnějšímu podporovanému mapperu. Jednotlivé stavy se díky vlastnostem typu union v paměti překrývají, což znemožňuje jejich současnou existenci (která v emulátoru nikdy nenastává) a výrazně snižuje celkovou paměťovou stopu objektu `Disk`.

V aktuální fázi vývoje emulátor implementuje podporu tří klíčových mapperů, které pokrývají podstatnou část knihovny her pro systém NES: NROM, UNROM a MMC3.

6.2 Sběrnice

Kromě propojovacího prvku, sběrnice slouží také jako správní centrum. Obsahuje frontu událostí, jako např. požadavek o načtení programu z SD karty nebo resetování, které zpracuje a odpovídajícím způsobem aktualizuje stav. Je velice riskantní provádět tento požadavek hned v okamžik přijetí požadavku, jelikož hrozí to poškozením stavu emulátoru. Proto byla navržena fronta událostí, která pomáhá odložit požadavky a zpracovat je v jeden, správný okamžik.

Když CPU přistupuje ke sběrnici, využívá funkce `bus_read()` a `bus_write()`, které realizují mapování systému NES.

```

void bus_write(Bus *self, uint16_t addr, uint8_t data) {
    /* Optimalizace: místo žebříku ifů je efektivnější rozdělit
     * adresový prostor na 4KB stránky */
    switch (addr >> 12) {
    case 0x0:
    case 0x1:
        self->memory[addr & 0x0fff] = data;
        break;
    case 0x2:
    case 0x3:
        /* Catch-up mechanismus: při přístupu k registrům, PPU dohání stav CPU. */
        ppu_run_until(&self->ppu, self->cpu.cycles * 3);
        ppu_write_reg(&self->ppu, 0x2000 + (addr & 7), data);
        break;
    case 0x4:
        if (addr == 0x4014) {
            handle_oam_dma(self, data);
        }
        break;
    }
    /* Ostatní mapování byly zkráceny... */
}

```

Výpis 2: Komunikační rozhraní sběrnice (příklad funkce pro zápis do adresního prostoru).

Naivní implementaci funkcí `bus_write()` a `bus_read()` mohl by být žebřík `ifů`, které postupně porovnávají parametr `addr` s adresovými rozsahy. Tato implementace však může být nepřátelská vůči branch predictor (který je obsazen i v RP2350) a také nutí procesor vykonávat posloupnost porovnání. Není garantováno, že kompilátor může být schopen optimalizovat.

Lepší návrh je využít toho faktu, že paměť NESu je fakticky rozdělena do stránek velkých 4096 bytů (0x1000). Například PPU je mapován do rozsahu 0x2000–0x3fff, což když vydělíme 4096, dostaneme rozsah 2–3. Pro rychlejší dělení je možné použít logický posun `addr >> 12`.

6.3 CPU

Pro reprezentaci registrů a adresaci pamětí se používají datové typy s fixní délkou – `uint8_t`, `uint16_t`, ..., které jsou dostupné v hlavičkovém souboru `stdint.h`. Nejen to pomáhá šetřit paměť, ale také přispívá k dokumentaci kódu.

Pro počítání cyklů se používá beznamínkový celočíselný datový typ `CycleCounter`, velikost kterého závisí na cílové architektuře – v případě RP2350 je to `uint32_t`, zatímco desktopový backend (architektura `x86_64`) používá `uint64_t`. Velikostí těchto typů odpovídají velikosti slova zmíněných architektur.

6.3.1 Vykonávací středisko CPU

CPU byl implementován jako interpret, jelikož procesor v NESu je poměrně jednoduchý a běží na taktovací frekvenci cca 1,7 MHz. Interpret je realizován na bázi podmíněčné konstrukce `switch` (výpis 3).

Pro zjištění adresovacího režimu dané instrukce při dekódování, se používá LUT tabulka adresovacích režimů, kde indexem je operační kód instrukce. Následně funkce `decode()` na základě

```

void cpu_step(Cpu *self) {
    if (self->nmi_pending) {
        cpu_handle_nmi(self);
        return;
    }
    /* Fetch a decode fáze */
    uint8_t opcode = read(self, self->PC++);
    uint16_t op_addr = decode(self, mode_table[opcode]);
    /* Execute fáze (interpret) */
    switch (opcode) {
    case OPCODE_AND_IMM:
        uint8_t data = read(self, op_addr);
        self->A &= data;
        SETZ(self->A == 0);
        SETN(self->A & 0x80);
        break;
    /* Zbytek instrukcí byl zkrácen... */
    }
}

```

Výpis 3: Krokovací funkce CPU.

adresovacího režimu zjistí adresu operandu.

Výpočet adresy operandu se řídí podle tabulky, popsané v [7], včetně počítání cyklů. Prvotní návrh emulátoru CPU obsahoval tabulku, kde pro každou instrukci byl přiřazen výsledný počet cyklů. Avšak při implementaci catch-up mechanismu pro PPU, to se stalo problémem: uprostřed dekódování CPU může zasahovat do registru PPU, a ten musí dohnat stav CPU v tento okamžik, ale když bude synchronizován podle cyklů procesoru před vykonáním instrukce, PPU bude opožděn vůči CPU o několik cyklů. Když PPU bude dohánět výsledný počet cyklu aktuální instrukce, ale instrukce ještě nebyla vykonaná, PPU bude napřed vůči PPU.

6.4 PPU

Funkce `ppu_run_until` (viz výpis 4 aktualizuje stav PPU a posouvá ho po cyklové ose dokud nedožene stav CPU (parametr `target_cycle`).

```

void ppu_run_until(Ppu *self, CycleCounter target_cycle) {
    while (self->cycles < target_cycle) {
        unsigned cycles_rem = SCANLINE_END - self->dot;
        unsigned cycles_to_run = target_cycle - self->cycles;
        unsigned step =
            (cycles_to_run < cycles_rem) ? cycles_to_run : cycles_rem;
        unsigned prev_dot = self->dot;
        unsigned next_dot = self->dot + step;

        /* Aktualizace stavu PPU... */

        self->cycles += step;
        self->dot = next_dot;
        /* Logika přechodu mezi scanline */
        if (self->dot >= 341) {
            /* inkrementace scanline, případné nastavení VBlank, ... */
        }
    }
}

```

Výpis 4: Funkce pro aktualizaci stavu PPU, implementovaná pomocí catch-up mechanismu.

Namísto inkrementální emulace po jednotlivých tečkách využívá funkce `ppu_run_until` principu skoků k významným událostem v rámci řádku. Algoritmus vypočítá vzdálenost k nejbližšímu bodu zájmu (např. cyklus odpovídající cyklu CPU nebo konec řádku) a provede aktualizaci stavu

v blocích. Tento přístup výrazně šetří instrukční cykly procesoru RP2350.

Když PPU dokončí vykreslování řádku, vyvolá callback, kde cílový kód může přímo poslat řádek na displej nebo provést další operace. To dovoluje PPU být nezávislým na způsobu vykreslování na cílové architektuře. Pixels se vykreslují do vnitřního bufferu.

6.4.1 Vykreslování pozadí

Proces generování obrazu v emulátoru musí být velice efektivní, aby RP2350 dokázal udržet stabilní snímkovou frekvenci. Funkce `render_bg` (výpis 5) realizuje vykreslování dlaždic pozadí s pixelovou granulací. Ačkoliv se data z paměti čtou po řádcích dlaždic (8 pixelů), samotný zápis do bufferu `scanline_buf` může probíhat po blocích. Tento přístup je také nezbytný pro správnou realizaci jemného horizontálního posunu (*fine X scrolling*).

```
void render_bg(Ppu *self, unsigned start, unsigned end) {
    unsigned curr = start;
    /* Výpočet počátečního offsetu v rámci první dlaždice */
    unsigned offset = (curr + self->x) & 7;
    uint8_t *dest = &self->scanline_buf[curr];
    while (curr < end) {
        /* Výpočet úseku, který zbývá vykreslit v aktuální dlaždici */
        unsigned rem = 8 - offset;
        unsigned count = (rem > (end - curr)) ? (end - curr) : rem;
        /* Načtení dat dlaždice a atributů z paměti */
        TileRow tile = fetch_tile(self);
        /* Bitový posun pro zarovnání pixelů podle jemného X posunu */
        tile >>= (offset << 2);
        /* Vnitřní smyčka pro zápis pixelů */
        for (unsigned i = 0; i < count; ++i) {
            *dest++ = tile & 0x0f; /* zápis 4bitové barvy */
            tile >>= 4;
            ++curr;
        }
        /* Další dlaždice již začínají od nultého pixelu */
        offset = 0;
        /* Kontrola přechodu na další dlaždici (hrubý X posun) */
        if (!((curr + self->x) & 7)) {
            increment_coarse_x(self);
        }
    }
}
```

Výpis 5: Funkce pro vykreslování dlaždicových řádku pozadí.

V rámci implementace byly použity tři klíčové optimalizační techniky:

- Logika jemného posunu je integrována přímo do kódu vykreslování pomocí bitových posunů. Tím odpadá nutnost dodatečného ořezávání a posunu obrazu.
- V kritické smyčce `for`, která zapisuje pixely do `scanline_buf`, se využívá pointerová aritmetika (`*dest++`) namísto přímého indexování pole pomocí `curr`. Slouží to jako optimalizační nápověda pro kompilátor, aby efektivněji využil registry a instrukce procesoru RP2350. Tím odpadá nutnost opětovného výpočtu adresy v každé iteraci smyčky.
- „Balení“ řádku dlaždice ve 32bitovém typu: tradiční reprezentace barev pomocí RGB v NES je pro moderní procesory nevýhodná. Byla využita technika balení osmi pixelů do jedné 32bitové proměnné (typ `TileRow`). Každý 4bitový nibble v této proměnné nese kompletní informaci o indexu palety i barvy daného pixelu. To umožňuje provádět jemný horizontální

posun pomocí jediného bitového posunu nad celým registrem, namísto osmi samostatných operací. Funkce `fetch_tile()`, která balí jeden řádek dlaždice do 32bitového typu, bude popsána v další sekci.

6.4.2 Příprava dat pro vykreslování pozadí

Funkce `fetch_tile()` (výpis 6) slouží k načtení celé dlaždice z VRAM podle adresy, nastavené interním 16bitovým registrem `v`.

```
TileRow fetch_tile(const Ppu *self) {
    uint16_t v = self->v; /* Aktuální adresní registr */
    /* 1. Výpočet adresy a načtení indexu dlaždice z Name Table */
    uint16_t addr = PPU_NT0 | (v & (VRAM_COARSE_X | VRAM_COARSE_Y | VRAM_NT_SEL));
    uint8_t tile_idx = self->nt[(addr >> 10) & 3][addr & 0x3ff];
    /* 2. Načtení atributů pro danou oblast */
    uint16_t attr_addr = calc_attr_addr(v);
    uint8_t attr = self->nt[(attr_addr >> 10) & 3][attr_addr & 0x3ff];
    /* 3. Výpočet posunu pro extrakci 2 bitů palety z bajtu atributů */
    unsigned shift = ((v >> 4) & 4) | (v & 2);
    uint8_t palette_idx = (attr >> shift) & 3;
    /* 4. Adresace Pattern Table a načtení dvou bitových rovin */
    uint16_t row_addr = get_pattern_table_bg(self) +
        (tile_idx << 4) + vram_get_fine_y(v);
    uint8_t row_low = self->chr[row_addr];
    uint8_t row_high = self->chr[row_addr + 8];
    /* 5. Syntéza 32bitového slova pomocí předpočítaných tabulek (LUT).
     * Transformace bitových rovin na rozprostřenou masku pixelů */
    uint32_t raw_pixels = row_high_lut[row_high] | row_low_lut[row_low];
    /* Aplikace indexu palety na všechny pixely v bloku */
    uint32_t attr_mask = attr_mask_lut[palette_idx];
    /* 6. Zabalení řádku do 32bitového typu TileRow (8 pixelů po 4 bitech) */
    return raw_pixels | attr_mask;
}
```

Výpis 6: Extrakce řádku dlaždice a následná jeho syntéza do 32bitového typu `TileRow` pomocí LUT.

Jak je patrné, využití LUT tabulek v této funkci slouží k úplnému zamezení redundantním výpočtům v kritické cestě emulátoru. Namísto procesorově náročné manipulace s jednotlivými pixely je transformace grafických dat redukována na několik operací čtení z paměti a bitových operací.

6.4.3 Vykreslování spritů

Po vykreslení bloku pixelů pozadí, proběhne algoritmus vykreslování spritů, který pracuje s jednotlivými sprity na řádku (pokud jsou, nebo vykreslování spritů je zapnuto). Funkce `render_sprites` (viz výpis 7) iteruje frontu spritů, řeší Sprite 0 Hit a prioritu vůči pozadí. Výsledné pixely zapisuje do `scanline_buf`.

Tato funkce řeší několik důležitých aspektů hardwaru NES:

1. Sprity jsou prohledávány v opačném pořadí – od nejnižší priority k nejvyšší.
2. Je respektována průhlednost pixelů spritu – pokud pixel spritu je průhledný, pixel v bufferu nebude ovlivněn.
3. Pokud se vykresluje první v pořadí sprite, pixel pozadí není průhledný a přitom pixel spritu

```

for (unsigned x = draw_start; x < draw_end; ++x) {
    uint8_t px = tile & 0x0f; /* Pixel spritu */
    uint8_t bg_px = *dest & 0x0f; /* Existující pixel pozadí v bufferu */
    bool bg_opaque = (bg_px & 0x03) != 0;
    /* 2. Je pixel spritu neprůhledný? */
    if ((px & 0x03) != 0) {
        /* 3. Detekce kolize pro Sprite 0 Hit */
        if (spr->is_0 && bg_opaque && x < 255) {
            self->stat |= PPU_STAT_ZERO_HIT;
        }
        /* 4. Zápis pixelu při splnění prioritních podmínek */
        if (!behind_bg || !bg_opaque) {
            /* Bit 4 značí, že jde o pixel (relevantní
             * při finálním vykreslování na obrazovku) */
            *dest = px | 0x10;
        }
    }
    ++dest;
    tile >>= 4;
}

```

Výpis 7: Kód pro vykreslení spritu s řešením priority pořadí, Sprite 0 Hit a průhledností.

není 255. pixel, nastane podmínka Sprite 0 Hit a příslušný bit v PPUSTAT bude nastaven (který pak přečte emulovaný program a zjistí, zda Sprite 0 Hit skutečně nastal).

4. Aby výsledný pixel spritu byl zapsán do bufferu, musí být vpředu (6. bit atributu spritu má být vypnut). Pokud není vpředu (odpovídající bit je zapnut), a přitom pixel pozadí není průhledný, pixel spritu bude zahozen.

6.4.4 Příprava dat pro vykreslování spritů

Princip funkce `fetch_spr_tile()` pro přípravu řádku dlaždice spritu pro vykreslování je stejný jako v případě řádku dlaždice pozadí. Vzhledem k tomu, že většina potřebné informace je již zakódována v OAM, výsledný algoritmus je značně jednodušší. Tato funkce je také založena na balení celého řádku dlaždice do 32bitového typu `TileRow` podobně jako `fetch_spr_tile()`, a aktivně využívá LUT tabulky, zejména pro převrácené řádky pixely, pokud je horizontální převrácení zapnuto.

Z důvodu zjednodušení se neřeší clipping. Toto však nemá zásadní vliv na základní funkčnost emulátoru.

6.4.5 Prohledávání OAM (Sprite Evaluation)

Jak bylo zmíněno v podkapitole 2.3.7, systém NES dokáže na jednom řádku zobrazit maximálně 8 spritů, a je nutné před samotným vykreslováním provést jejich výběr. Funkce `queue_sprites` (výpis 8) prochází paměť OAM a identifikuje objekty, které protínají aktuálně emulovaný scanline.

```

for (unsigned i = 0; i < 64 && self->queued_sprites_count < 8; ++i) {
    uint8_t y = self->oam[i * 4];
    if (y >= 239) continue; /* Sprite mimo viditelnou oblast */

    unsigned row = self->scanline - y;
    if (row < h) { /* Sprite protíná aktuální řádek */
        Sprite *spr = &self->queued_sprites[self->queued_sprites_count];
        uint8_t tile = self->oam[i * 4 + 1];
        uint8_t attr = self->oam[i * 4 + 2];
        uint16_t pat_addr;

        spr->pos_y = y;
        spr->pos_x = self->oam[i * 4 + 3];
        spr->attr = attr;
        spr->is_0 = (i == 0); /* Důležité pro detekci Sprite 0 Hit */

        /* Výpočet jemné Y souřadnice s ohledem na vertikální převrácení */
        unsigned y_fine = (attr & 0x80) ? (h - 1 - row) : row;

        /* Výpočet adresy v Pattern Table */
        if (h == 8) { /* Režim 8x8 */
            uint16_t base = get_pattern_table_spr(self);
            pat_addr = base + (tile * 16) + y_fine;
        } else { /* Režim 8x16 (bit 0 určuje banku, zbytek index) */
            uint16_t base = (tile & 1) ? 0x1000 : 0x0000;
            uint8_t tile_idx = tile & 0xfe;
            if (y_fine >= 8) {
                ++tile_idx;
                y_fine -= 8;
            }
            pat_addr = base + (tile_idx * 16) + y_fine;
        }

        /* Přednačtení dat dlaždice pomocí LUT (stejně jako u pozadí) */
        spr->tile = fetch_spr_tile(self, pat_addr, attr);
        ++self->queued_sprites_count;
    }
}

```

Výpis 8: Implementace Sprite Evaluation. Funkce připravuje data pro 8 objektů na aktuálním řádku.

Klíčové aspekty implementace:

- Smyčka prochází všech 64 spritů (256 bytů) v paměti OAM, ale ukončí se ihned po nalezení prvních osmi platných objektů. Tim je emulován limit původního hardwaru a zároveň šetřen procesorový čas mikrokontroléru.
- Tato implementace bere v povahu to, že systém NES měl sprity posunute o jeden pixel (viz 2.3.7).
- Funkce dynamicky přepíná mezi standardním režimem 8×8 a rozšířeným 8×16 podle registru PPU_CTRL. V režimu 8×16 funguje adresování odlišně – nultý bit indexu dlaždice slouží jako volič banky (Pattern Table), zatímco zbylé bity určují konkrétní dvojici dlaždic.
- Pokud je v attributech nastaven příznak pro vertikální převrácení (0x80), index řádku uvnitř dlaždice se invertuje. Tato operace probíhá před výpočtem adresy v CHR paměti (pokud by převrácení se aplikovalo při vykreslování, značně by to zkomplikovalo algoritmus a také výpočetní výkon).

6.5 APU

Emulace subsystému APU byla nejtěžší etapou ve vývoji emulátoru, jelikož ladit zvuk není tak jednoduchý jako grafický výstup – v případě PPU jakékoliv nesrovnatelnosti jsou snadno pozorovatelné a ve většině času příčina chyby je přibližně jasná. V případě zvuku je to poněkud komplikovanější. Proto většinu času při emulaci APU bylo stráveno laděním a opětovným přečtením dostupné dokumentace.

Nejdříve byl implementován pulzní kanál, protože je nejjednodušší a nejpoužívanější v demonstračních hrách. Postupně se implementovaly ostatní kanály. Desktopový backend využívá zvukový API knihovny SDL3, zatímco backend RP2350 využívá vlastní zvukový ovladač, který bude popsán v sekci 10.2.

6.5.1 Vzorkování (sampling)

Jak již bylo zmíněno v 1.7.3, je potřeba provést downsampling v případě jestli zvuková jednotka generuje vzorky s příliš velkou frekvencí, což je případ APU v NESu.

Použijeme metodu decimace, která spočívá ve filtraci signálu pomocí dolní propustí a následném výběru N -tého vzorku. Naše implementace ale zvolila metodu jednodušší decimace – metodu přímého výběru každého N -tého vzorku bez dolní propustí. Vzhledem k nelineární povaze a nízkému rozlišení původního zvukového čipu APU je výsledný rozdíl v kvalitě zvuku pro běžného uživatele téměř nepostřehnutelný.

Pro realizaci výběru N -tého vzorku, je uchováván akumulátor času mezi vzorky. Při každém vydaném vzorku se k akumulátoru přičte čas. Problémem ale je, že čas mezi jednotlivými vzorky nemusí být celočíselný – závisí na taktovací frekvenci emulovaného CPU f_{CPU} (pro systém NES platí $f_{CPU} \approx 1,79$ MHz) a zvolenem sample ratu f_s , což lze vyjádřit vztahem:

$$step = \frac{f_{CPU}}{f_s}$$

Pro RP2350 backend platí $f_s = 31,25$ kHz (desktop backend používá standardní frekvenci vzorkování 44,1 kHz). V případě backendu RP2350 vychází tento krok přibližně na 57,1786 cyklů. Proto použití zaokrouhlování pro akumulátor času by vedlo k akumulaci chyby v časování, což by se projevilo slyšitelným kolísáním výšky tónu nebo praskáním vlivem nesprávné fáze signálu.

Aby bylo možné dosáhnout optimální přesnosti bez použití výpočetně náročné aritmetiky s plovoucí čárkou, využívá implementace **aritmetiku s pevnou řádovou čárkou** (*fixed-point*) ve formátu 16.16. V tomto formátu je 32bitové slovo rozděleno na dvě poloviny:

- **celočíselnou část** – představuje celočíselný počet cyklů.
- **zlomkovou část** – uchovává desetinnou část s přesností na $\frac{1}{65536}$.

6.5.2 Krokování a generování vzorků

Funkce `apu_run_until` (viz výpis 9) aktualizuje stav APU a posouvá ho po cyklové ose dokud nedožene stav CPU (parametr `target_cycle`).

```
void apu_run_until(Apu *self, CycleCounter target_cycle) {
    CycleCounter cycles_to_run = target_cycles - self->cycles;

    while (cycles_to_run >= 2) {
        /* Aktualizace všech kanálů (trojúhelníkový kanál se
         * aktualizuje 2krát, jelikož je taktován stejnou frekvencí
         * jako CPU) */
        update_triangle(&self->triangle);
        update_triangle(&self->triangle);
        update_pulse(&self->p1);
        update_pulse(&self->p2);
        update_noise(&self->noise);
        update_frame_counter(self);
        /* Downsampling: vzorek se generuje jen po překročení
         * určitého času (po poslední generaci) */
        self->sample_acc += (2 << 16); /* +2 cyklů CPU */
        if (self->sample_acc >= self->sample_step) {
            self->sample_acc -= self->sample_step;
            output_sample(self);
        }

        cycles_to_run -= 2;
        self->cycles += 2;
    }
}
```

Výpis 9: Aktualizace stavu APU.

Při každém kroku APU se aktualizují vnitřní stav a výstup každého kanálu. Následně se ověřuje, zda je na čase vygenerovat nový vzorek – jakmile akumulátor `sample_acc` dosáhne prahové hodnoty `sample_step`, je vygenerován výstupní vzorek a z akumulátoru je odečtena délka kroku, čímž je zajištěna fázová spojitost signálu bez kumulativní chyby v časování.

Ačkoliv je APU taktováno stejnou frekvencí jako CPU, většina jeho komponentů je aktualizována pouze každý druhý cyklus CPU. V rámci implementace je proto výhodné zpracovávat APU po blocích dvou CPU cyklů, přičemž komponenty běžící na plné frekvenci (trojúhelníkový kanál) jsou aktualizovány dvakrát v rámci jednoho kroku.

Aby bylo možné minimalizovat drahé operace dělení a aritmetiku plovoucí čárky, vzorky se generují pomocí LUT tabulky (viz [13]). Tyto LUT tabulky se inicializují při prvotní inicializaci APU (výpis 10).

6.6 BIOS

BIOS představuje první software, který emulovaný CPU po spuštění vykoná. V tomto projektu neplní BIOS pouze funkci inicializace hardwaru, ale slouží jako grafické uživatelské rozhraní pro výběr programů z SD karty.

Hlavní motivací pro tvorbu vlastního BIOSu bylo poskytnout uživateli jednoduchý a přívětivý způsob k výběru programu pro načtení a konfiguraci zařízení. Řešením by bylo vytvořit interaktivní menu, kde uživatel bude moci procházet souborový systém SD karty nebo přejít do režimu

```

void apu_init(Apu *self, Bus *bus) {
    /* Zkráceno... */
    for (int i = 0; i < 31; ++i) {
        pulse_lut[i] = (95.52 / (8128.0 / (double) i + 100.0)) *
                      APU_SAMPLE_MAX_VALUE;
    }
    for (int i = 0; i < 203; ++i) {
        tnd_lut[i] = (163.67 / (24329.0 / (double) i + 100.0)) *
                   APU_SAMPLE_MAX_VALUE;
    }
}

void output_sample(Apu *self) {
    /* Zkráceno (ověřování jestli jednotlivé kanály jsou zapnuté
     * nebo vypnuté)... */
    ApuSample pulse_smp = pulse_lut[p1_out + p2_out];
    ApuSample tnd_smp = tnd_lut[3 * tri_out + 2 * noise_out + dmc_out];

    self->sample_cb(self, pulse_smp + tnd_smp);
}

```

Výpis 10: Generace vzorků pomocí LUT tabulky, která má předvypočítané hodnoty pomocí lineární aproximace popsané v [5].

nastavení.



Obrázek 6.1: Ukázka hlavního menu BIOS.

Pro realizaci BIOSu byl použit kompilátor cc65, který umožňuje psát kód v jazyce C. Použití assembleru 6502 pro komplexnější logiku by bylo časově náročné a náchylné k chybám. Také byla použita knihovna neslib, která poskytuje optimalizované abstrakce pro práci s paletami a vykreslováním, sprity a čtení stavu herních ovladačů.

Ve výpisu 11 je uvedena zkrácená struktura kódu BIOSu. Využívá to typickou strukturu pro NES programy:

1. Inicializace subsystemu
2. Hlavní smyčka programu:
 - (a) Synchronizace s obrazem pomocí V-Blank

(b) Přechtení vstupu

(c) Logika programu

```
#include "neslib.h"
#include "fwx.h"
/* 6502 má mnohem menší kapacitu zásobníku, proto je lepší
 * uchovávat většinu stavu programu v interní RAM. */
static uint8_t pad1;
static uint8_t pad1_last;
static uint8_t pad1_pressed;
/* ... */
void main(void) {
    /* Inicializace grafiky */
    ppu_off();
    pal_bg(palette_bg);
    pal_spr(palette_bg);
    bank_spr(0);

    clear_screen();
    draw_header();

    /* Inicializace FWX (viz dále)... */

    while (true) {
        /* Synchronizace s monitorem (V-Blank) */
        ppu_wait_nmi();
        /* Přechtení vstupu z ovladače */
        read_input();

        if (pad1_pressed & PAD_DOWN) {
            move_cursor(+1);
        } else if (pad1_pressed & PAD_UP) {
            move_cursor(-1);
        } else if /* Zkráceno... */

    }
}
```

Výpis 11: Struktura kódu BIOS (zkráceno)

Klíčovým v kódu BIOSu je využití našeho protokolu FWX (viz následující sekce 6.6). Normálně NES programy nemohou vystoupit z rámce emulovaného systému NES, aby například přistupovaly k SD kartě. FWX je založen na modelu *příkaz-odpověď*, když emulovaný program pošle příkaz a dostane odpověď. Přitom veškerou komunikaci zahajuje emulovaný program, nikoliv emulator.

```
void main(void) {
    /* ... */

    /* Zahajení komunikace s FWX */
    fwx_start();
    /* Posílání příkazu o zjištění počtu souborů */
    fwx_write_u8(FWX_CMD_SD_FILE_COUNT);
    /* Přechtení výsledků a uložení */
    file_count = fwx_read_u8();
    /* Zastavení komunikace */
    fwx_stop();

    /* ... */
}
```

Výpis 12: Příklad využití komunikačního rozhraní FWX v kódu BIOSu.

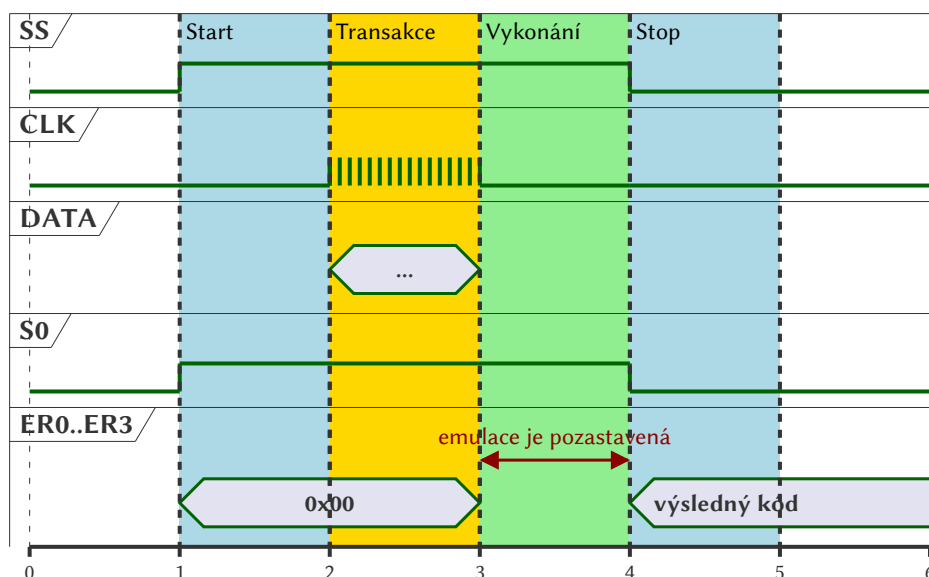
Ve výpisu 12, BIOS zahajuje komunikaci pomocí funkce `fwx_start()`. Pro zobrazení souborů, BIOS nejdříve potřebuje vědět jejich skutečný počet v současné složce, takže pošle požadavek pomocí funkce `fwx_write_u8()`, která zapisuje jeden byte do datového registru FWX. Po vykonání příkazu emulátor vrátí počet souboru jeho uložení do datového registru. BIOS pak je schopen přečíst tyto data pomocí `fwx_read_u8()` (tato funkce konkrétně čte jeden byte) a následně

zastaví komunikaci pomocí funkce `fwx_stop()`.

Momentálně BIOS je schopen zobrazovat max. 255 souborů v jedné složce, a umí jenom vypisovat soubory a načítat program. V budoucnu se plánuje realizovat rozsáhlé menu nastavení a vylepšit BIOS vzhledově.

6.7 FWX (FirmWare eXchange)

Aby mohl emulovaný systém (např. BIOS) přistupovat k hardwarovým prostředkům mimo standardní architekturu NES – jako je souborový systém na SD kartě nebo konfigurace zařízení – bylo nutné implementovat komunikační most, nazvaný **FWX**. Toto rozhraní umožňuje emulovanému softwaru bezpečně předávat požadavky hostitelskému firmwaru emulátoru, a to prostřednictvím trojice paměťově mapovaných registrů (**FWXCTRL**, **FWXSTAT** a **FWXDATA**).



Obrázek 6.2: Zjednodušené schéma typického průběhu komunikace přes rozhraní FWX.

Rozhraní FWX je navrženo jako jednoduchý, bajtově orientovaný synchronní protokol založený na modelu **příkaz-odpověď** (*command-response*). Veškerou komunikaci iniciuje výhradně emulovaný program (BIOS).

Rámcování zpráv je řízeno pomocí kontrolního bitu **SS** (Start/Stop) v řídicím registru **FWXCTRL**. Příkazy a jejich datové argumenty jsou přenášeny přes datový registr **FWXDATA**. K zachycení zapisovaných dat emulátorem dochází při explicitní změně stavu hodinového bitu **CLK** (data latching). Vzhledem k tomu, že je protokol taktován softwarově samotným emulovaným programem, nejsou zde kladeny žádné striktní požadavky na časování mezi jednotlivými hodinovými událostmi.

Z pohledu emulovaného programu je exekuce příkazů blokující. Po odeslání požadavku je běh emulace dočasně pozastaven, dokud hostitelský firmware příkaz nezapracuje. Dokončení operace

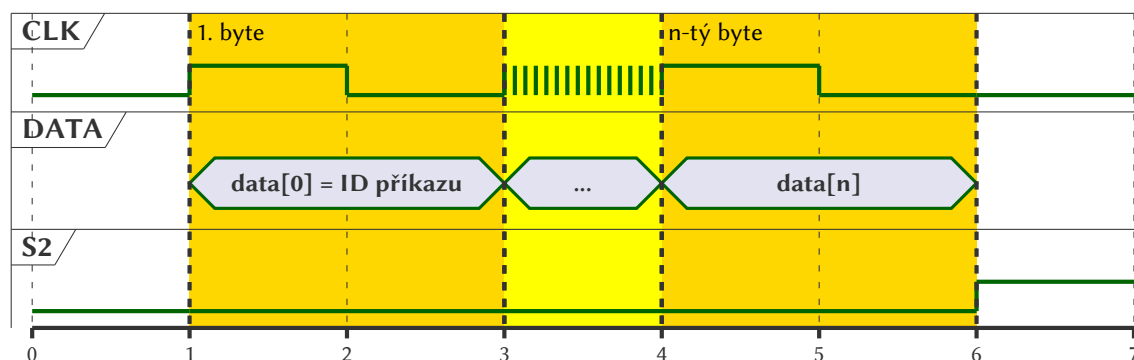
je následně signalizováno příznakem *S0* ve stavovém registru **FWXSTAT**.

6.7.1 Iniciační a terminační sekvence (START/STOP)

Komunikační průběh je striktně ohraničen stavy START a STOP, které generuje emulovaný program.

- **START podmínka** je vyvolána změnou bitu *SS* z logické 0 na logickou 1. Po jejím vydání emulátor nastaví stavový bit *S0* na 1 (indikace probíhající komunikace) a zároveň vynuluje příznaky *S1*, *S2* a chybové kódy *ER0..ER3*.
- **STOP podmínka** nastává při změně bitu *SS* z 1 na 0. Emulátor vynuluje stavové bity *S0*, *S1* a *S2*, přičemž bity *ER0..ER3* si zachovají chybový kód poslední provedené operace pro případnou kontrolu ze strany BIOSu.

6.7.2 Přenos dat a příkazů



Obrázek 6.3: Detailní stavový diagram procesu odesílání dat z emulovaného programu do emulátoru.

Samotný přenos informací směrem k emulátoru probíhá po iniciaci START podmínky. Emulovaný program zapíše datový byte do registru **FWXDATA** a následně nastaví bit *CLK* v registru **FWXCTRL** na logickou 1. Jakmile emulátor data zpracuje, okamžitě bit *CLK* automaticky resetuje na 0, čímž potvrdí úspěšný příjem.

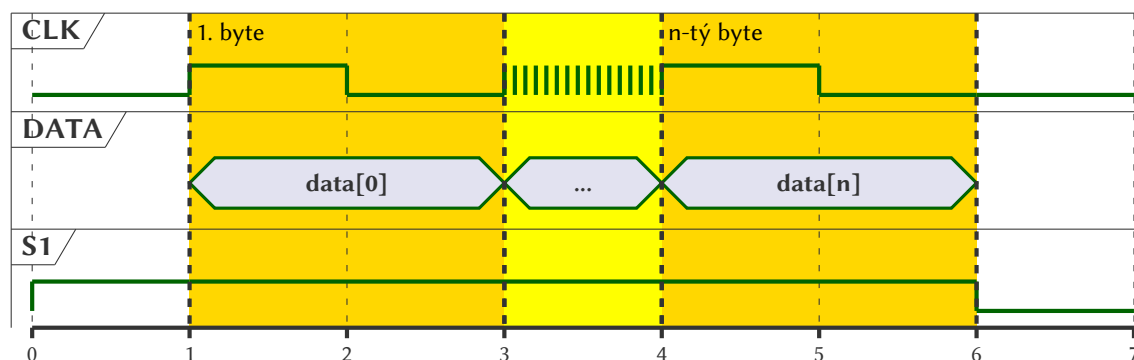
První byte odeslaný po START podmínce je vždy interpretován jako **ID příkazu** (Command ID). Pokud daný příkaz vyžaduje dodatečné argumenty (např. název souboru nebo index), musí být tyto byty odeslány bezprostředně po ID příkazu. Dokud emulátor neobdrží očekávaný počet datových bytů, udržuje stavový bit *S2* na hodnotě 1, čímž si žádá další data.

6.7.3 Zpracování příkazu (blokující exekuce)

Po úspěšném přijetí ID příkazu (a všech případných argumentů) zahájí hostitelský firmware jeho exekuci. Během této fáze je emulace pozastavena. Tím je zaručena synchronizace – emulo-

vaný program se po provedení instrukce zápisu probudí přesně v okamžiku, kdy je operace na straně emulátoru dokončena. Výsledek operace je reflektován prostřednictvím chybových kódů v registru **FWXSTAT** (viz sekce 6.8.1).

6.7.4 Příjem návratových dat



Obrázek 6.4: Detailní stavový diagram procesu příjmu návratových dat emulovaným programem.

Některé příkazy (např. čtení obsahu adresáře) generují data, která musí být předána zpět emulovanému programu. Přítomnost těchto návratových dat je signalizována nastavením bitu *S1* na hodnotu 1.

Proces příjmu je inverzní k odesílání: emulovaný program si přečte hodnotu z registru **FWX-DATA** a pro vyžádání dalšího bajtu musí zapsat logickou 1 do bitu *CLK*. Po obdržení dalšího bajtu, bit *CLK* je automaticky nastaven na 0. Jakmile jsou všechna návratová data úspěšně přečtena, emulátor nastaví bit *S1* zpět na 0.

6.7.5 Řetězení příkazů

Komunikační protokol je navržen s ohledem na efektivitu; pokud je exekuce příkazu dokončena a na lince je stále aktivní úroveň *SS* (hodnota 1), může emulovaný program plynule navázat odesláním dalšího ID příkazu bez nutnosti opětovně provádět STOP a START sekvenci.

6.8 Mapování a přehled registrů

Adresa	Název	Směr	Účel
\$4018	FWXSTAT	R	Stavový registr a chybové kódy.
\$4019	FWXCTRL	R/W	Řízení komunikace.
\$401A	FWXDATA	R/W	Obousměrný datový registr.

Tabulka 6.1: Přehled komunikačních registrů rozhraní FWX.

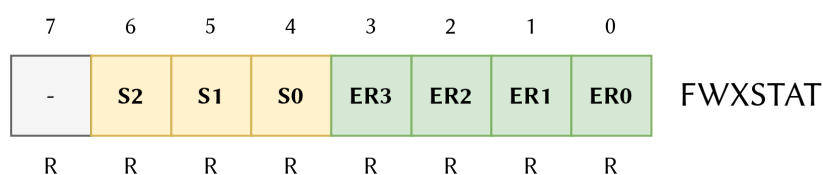
Pro zajištění komunikace mezi emulovaným programem a emulátorem je rozhraní FWX mapováno přímo do adresového prostoru CPU. V původním systému NES byl paměťový blok \$4018-\$401F vyhrazen pro testovací režimy APU a CPU, avšak v produkčních verzích konzole byla tato funkčnost deaktivována (tudíž není ani potřeba tuto funkčnost emulovat). Proto tento nevyužitý prostor je bezpečně recyklován pro své komunikační registry, čímž předchází konfliktům se standardními hrami (je ovšem nezbytné se vyhnout mapperům, které by tento prostor nestandardně využívaly).

Samotné rozhraní FWX se skládá ze tří 8bitových registrů, přičemž zbylý adresní prostor (\$401B-\$401F) je rezervován pro budoucí rozšíření protokolu (např. plánované rozhraní pro přímý přístup do paměti).

6.8.1 Detailní popis registrů

Stavový registr FWXSTAT (\$4018)

Tento registr slouží výhradně ke čtení ze strany emulovaného programu a poskytuje zpětnou vazbu o stavu protokolu a případných chybách běhu. Výchozí hodnota po resetu je 0x00.



Obrázek 6.5: Rozvržení registru FWXSTAT.

Stavové bity (S0–S2):

- **S0 (probíhá přenos)** je nastaven na logickou 1 po celou dobu trvání přenosu (mezi podmínkami START a STOP).
- **S1 (data připravena)** je nastaven na logickou 1 v případě, že emulátor po úspěšném zpracování příkazu vrací data, která je možné vyčíst z registru FWXDATA.
- **S2 (přenos argumentů)** indikuje, zda emulátor očekává další datové bajty pro aktuální příkaz. Dokud je **S2** rovno 0, probíhá sběr datových bajtů. Jakmile je obdržen potřebný počet bajtů a příkaz je proveden, je bit **S2** nastaven na 1. Příznak je automaticky vymazán při následném čtení registru nebo při vydání STOP podmínky.

Bit 7 je v současné verzi rozhraní rezervován.

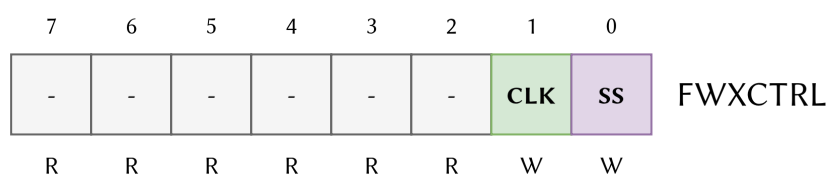
Chybové kódy (ER0–ER3): spodní čtyři bity uchovávají chybový kód poslední provedené operace. Emulovaný program by měl tyto bity kontrolovat po každém ukončeném příkazu. Seznam možných chyb je uveden v tabulce 6.2.

Hodnota	Mnemotechnika	Popis chyby
0x0	ERR_OK	Operace proběhla úspěšně.
0x1	ERR_BAD_DATA	Přijata neplatná struktura dat nebo neznámý příkaz.
0x2	ERR_SD_MISSING	SD karta není vložena nebo selhala inicializace.
0x4	ERR_SD_IO_ERROR	Fyzická I/O chyba při práci se souborovým systémem.
0x5	ERR_BAD_IDX	Index mimo rozsah v aktuálním pracovním adresáři.
0x6	ERR_NON_EXISTENT	Požadovaný soubor nebo složka neexistuje.
0x7	ERR_IS_FILE	Cílový objekt je soubor (očekáván adresář).
0x8	ERR_IS_DIR	Cílový objekt je adresář (očekáván soubor).

Tabulka 6.2: Přehled chybových kódů vrácených v registru FWXSTAT.

Řídicí registr FWXCTRL (\$4019)

Slouží pro obousměrnou manipulaci s kontrolními signály linky. Zápisem do tohoto registru emulovaný program fyzicky taktuje protokol. Výchozí hodnota je 0x00.



Obrázek 6.6: Rozvržení registru FWXCTRL.

- **SS (Start/Stop)** řídí životní cyklus komunikace. Zápis logické 1 otevírá relaci (START), zápis logické 0 relaci ukončuje (STOP).
- **CLK (Clock / Latch)** funguje jako softwarové hodiny protokolu.
 - Při odesílání dat emulátoru (zápis do FWXDATA) se nastavením **CLK** na 1 data *uzamknou* (latch) a odešlou.
 - Při čtení dat z emulátoru se nastavením **CLK** na 1 vyžádá další bajt odpovědi.

Změna bitu na 1 je ze strany emulátoru detekována ihned. Po zpracování (což zastaví cyklus procesoru) je bit **CLK** emulátorem automaticky vymazán zpět na 0, čímž je operace potvrzena a emulace může pokračovat.

Bity 2 až 7 jsou rezervovány.

Datový registr FWXDATA (\$401A)

Tento registr (čtení/zápis) slouží jako fyzické médium pro přesun samotných dat – tedy ID příkazů, jejich argumentů a návratových hodnot. Práce s tímto registrem je vždy podmíněna následnou manipulací s bitem **CLK** v registru FWXCTRL. Výchozí hodnota je 0x00.

6.9 Sada příkazů

Protokol FWX definuje sadu příkazů, pomocí kterých emulovaný program přistupuje k externím prostředkům. Pro potřeby BIOSu byly implementovány především příkazy pro manipulaci se souborovým systémem na SD kartě a pro řízení indikačních LED diod zařízení.

Veškerá data přenášená přes rozhraní FWX využívají následující datové typy:

- `uint8_t` – standardní 8bitový bezznaménkový bajt.
- `cstring` – textový řetězec znaků v kódování ASCII zakončený nulovým znakem (0x00), odpovídající standardu jazyka C.

6.9.1 Přehled podporovaných příkazů

Tabulka 6.3 shrnuje dostupné příkazy protokolu FWX. Očekávané argumenty (*Data*) musí být odeslány bezprostředně po ID příkazu. Návratové hodnoty (*Návrat*) je nutné z emulátoru vyčíst před ukončením komunikace.

Poznámka k chybovým stavům: možné chyby (např. neexistující soubor `ERR_NON_EXISTENT` nebo odpojená SD karta `ERR_SD_MISSING`) nejsou pro úsporu místa v tabulce 6.3 explicitně vypsané. Emulovaný program musí po každém příkazu ověřit chybový kód v registru FWXSTAT.

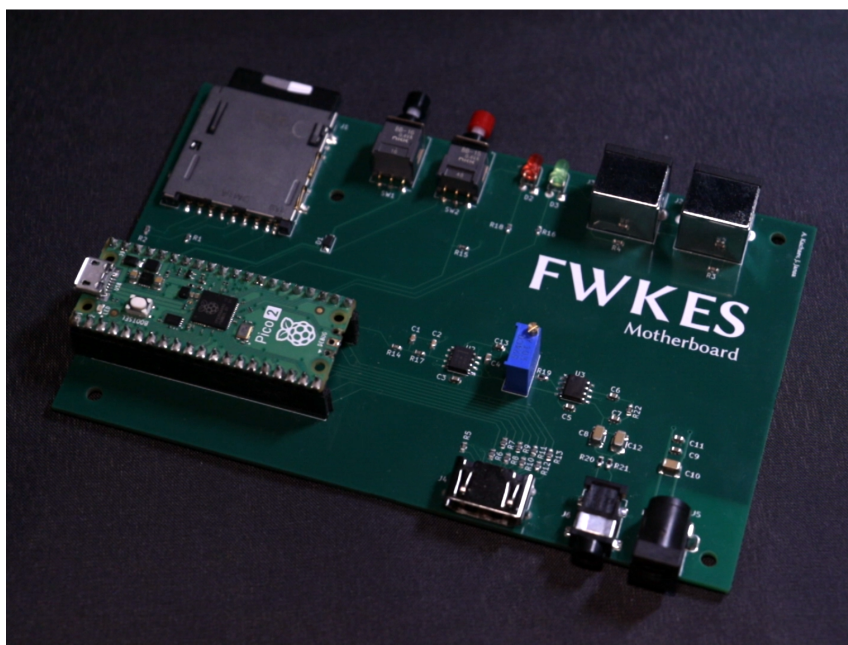
ID	Mnemotechnika	Argumenty (Data)	Návrat	Popis
0x00	RESET	žádné	žádný	Kompletní reinicializace emulátoru, vynulování RAM a znovunačtení BIOSu.
0x01	SD_LOAD	1. uint8_t (délka) 2. cstring (cesta)	žádný	Načte a spustí .nes ROM soubor. Pokud je cílem složka, změni do ní aktuální pracovní adresář.
0x02	SD_LOAD_IDX	1. uint8_t (index)	žádný	Analogické k 0x01, ale cíl je specifikován indexem v aktuálním adresáři.
0x03	SD_FILENAME_IDX	1. uint8_t (index)	cstring (název)	Vrátí název souboru na zadaném indexu. Neexistuje-li, vrátí prázdný řetězec.
0x04	SD_IS_DIR	1. cstring (cesta)	uint8_t (bool)	Ověří, zda zadaná cesta ukazuje na složku (vrací 1) nebo soubor (vrací 0).
0x05	SD_IS_DIR_IDX	1. uint8_t (index)	uint8_t (bool)	Analogické k 0x04, ale cíl je specifikován indexem.
0x06	SD_PWD	žádné	cstring (cesta)	Vrátí absolutní cestu k aktuálnímu pracovnímu adresáři emulátoru.
0x07	SD_CWD	1. cstring (cesta)	žádný	Změni aktuální pracovní adresář emulátoru na zadanou cestu.
0x08	SD_FILE_COUNT	žádné	uint8_t (počet)	Vrátí celkový počet položek (souborů a složek) v aktuálním pracovním adresáři.
0x09	LED_ERROR	1. uint8_t (stav 1/0)	žádný	Zapne (1) nebo vypne (0) červenou chybovou indikační LED na šasi zařízení.
0x0A	LED_GENERAL	1. uint8_t (stav 1/0)	žádný	Zapne (1) nebo vypne (0) zelenou indikační LED (používáno např. při načítání).

Tabulka 6.3: Přehled příkazů FWX rozhraní. Datové typy argumentů a návratových hodnot jsou uvedeny v závorkách.

Kapitola 7

Návrh základní desky

Tato kapitola popisuje návrh základní desky systému, včetně schématu, desky plošných spojů a krytu. Zaměřuje se na jednotlivé funkční bloky, jako je obrazový výstup, zvukový subsystém, úložiště dat, připojení vstupních zařízení a napájení.



Obrázek 7.1: Vyrobená základní deska.

Celý hardware byl navrhován s důrazem na stabilitu, čistotu signálu a dlouhodobou spolehlivost. Kompletní návrh schématu i desky plošných spojů (DPS) byl vypracován v open-source programu KiCad. Jedním z problémů bylo integrovat mikrokontrolér spolu s citlivými analogovými (audio) a vysokofrekvenčními obvody (HDMI a SD karta) na omezeném prostoru. Celé elektrické schéma zapojení základní desky a návrh DPS je možné najít v přílohách A a B.

7.1 Napájení

Celý systém, včetně ovladačů, je napájen od 5 voltů skrz standardní koaxiální napájecí konektor s průměrem 5.5mm, který používají spotřebitelské adaptéry 5V DC. Díky tomu lze zařízení připojit do elektrické sítě pomocí standardního adaptéru.

Pro zvětšení stability napájení a zjednodušení návrhu, DPS byla navržena jako 4vrstvá deska, což znamená, že má 2 signálové vrstvy na přední a zadní straně, a 2 napájecí vrstvy uvnitř pro VCC a GND. Díky tomu lze výrazně zkrátit délku napájecích cest a zvýšit stabilitu napájení a logiky.

7.2 Video výstup HDMI

Ačkoliv mikrokontrolér RP2350 nedisponuje nativním hardwarovým řadičem pro HDMI, emulátor generuje digitální obrazový signál pomocí softwarově definovaného protokolu na bázi DVI, který je zpětně-kompatibilní s HDMI, jelikož oba využívají technologii TMDS pro přenos video dat. Video signal z mikrokontroléru je fyzicky vyveden přes standardní HDMI konektor.

Signálové piny GPIO mikrokontroléru jsou k HDMI konektoru připojeny přes sériové rezistory o hodnotě 220 Ω . Toto řešení vychází z obvodu adaptéru Adafruit DVI Breakout, který jsme zpočátku používali, a je založen na informaci z oficiální specifikace DVI, sekce *T.M.D.S. Electrical Specification*.^[15]

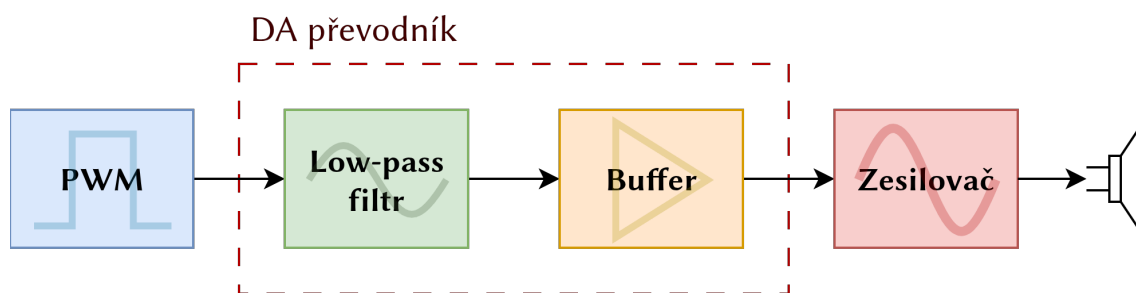
Při integrování na plošný spoj byla věnována pozornost trasování těchto vysokofrekvenčních signálů. Obrazová data jsou vedena jako diferenciální páry (tři datové a jeden hodinový kanál). Pro zachování fázové synchronizace barevných složek musely být délky jednotlivých vodičů v páru i párů mezi sebou striktně vyrovnány. Všechny tyto trasy navíc vedou nepřerušovaně nad vnitřní zemní rovinou, aby byla dodržena charakteristická diferenciální impedance 100 Ω ^[15].

7.3 Rozhraní SD karty

Pro ukládání herních ROM souborů a uživatelských dat je na desce integrován slot pro paměťovou kartu. Z elektrického hlediska je klíčovou součástí tohoto obvodu soustava *pull-up* rezistorů (typicky 10 k Ω), kterými jsou osazeny linky *CMD* a *DAT0* linky. Aplikace těchto rezistorů přímo reflektuje požadavky standardu *SD Physical Layer Specification*.^[16] Během inicializace nebo ve fázích nečinnosti (kdy je sběrnice ve stavu vysoké impedance) zajišťují tyto rezistory, že signály neplavou (jsou v nedefinovaném stavu), ale drží stabilní úroveň logické jedničky (3,3 V). Tím se účinně předchází falešným detekcím startu komunikace vlivem okolního elektromagnetického rušení (EMI). Navíc kryt konektoru SD karty (shielding) je připojen k zemi, což zajišťuje ochranu proti EMI a elektrostatickému náboji při vložení SD karty.

7.4 Analogový audio výstup

Jelikož mikrokontrolér RP2350 nedisponuje integrovaným DA převodníkem, je zvukový signál z emulovaného APU generován pomocí pulzně šířkové modulace (PWM) na frekvenci 250 kHz. Aby bylo možné tento digitální signál reprodukovat v běžných audio zařízeních, musí projít procesem rekonstrukce a zesílení (viz blokové schéma na obrázku 7.2).



Obrázek 7.2: Blokové schéma analogové částí audio výstupu.

7.4.1 Rekonstrukce a předzesílení

V první fázi je PWM signál veden přes pasivní dolní propust (rekonstrukční filtr). Ten potlačuje vysokofrekvenční nosnou složku a vyfiltruje užitečnou audio obálku. Následuje aktivní stupeň s operačním zesilovačem zapojeným jako napěťový sledovač. Tento blok plní dvě úlohy:

- Impedanční oddělení – chrání výstupní piny mikrokontroléru před zátěží a stabilizuje signál.
- Aktivní filtrace – zajišťuje strmější odříznutí šumu a zbytků PWM frekvence, čímž zvyšuje čistotu zvuku.

7.4.2 Výkonové zesílení a výstup

Pro řízení úrovně hlasitosti je v cestě zařazen kalibrační trimr, který slouží jako pasivní dělič napětí. Finální zesílení signálu na úroveň vhodnou pro sluchátka zajišťuje integrovaný audio zesilovač LM386. Ten je konfigurován tak, aby poskytoval dostatečnou amplitudu při zachování stability i při kapacitní zátěži.

Výstupní řetězec je zakončen vazebními kondenzátory, které eliminují stejnosměrnou (DC) složku, a ochrannými rezistory proti zkratu. Audio výstup je vyveden na standardní 3,5 mm jack konektor a je dimenzován pro sluchátka s impedancí 32–600, Ω . Podrobný popis jednotlivých komponentů a kompletní schéma zapojení jsou součástí přílohy E.

7.5 Mikrokontrolér

Pro snadnou výměnu mikrokontroléru (respektive vývojové desky Raspberry Pi Pico 2) a zároveň jednodušší návrh, plošný spoj je vybaven hnízdem, kam se vkládá vývojová deska. Přímá integrace mikrokontroléru RP2350 a okolního obvodu by ztížilo celkový návrh, zvýšilo by cenu výroby a zároveň by neumožnilo používat neoficiální vývojové desky (které mohou např. rozšiřovat RAM paměť). Navíc overclocking zkracuje životnost mikrokontroléru, proto uživatel bude moci vyměnit mikrokontrolér v případě poruchy.

Tento přístup také umožňuje použití jiných desek založených na mikrokontroléru RP2350. Tak například je možné použít desku Pimoroni Pico Plus 2, která nabízí 8 MB SRAM a 16 MB Flash paměti navíc, a také USB-C konektor.

Kapitola 8

Návrh ovladačů

Navržený obvod (viz přílohy C a D) představuje digitální vstupní rozhraní určené pro snímání stavů osmi mechanických spínačů a jejich sériový přenos do mikrokontroléru prostřednictvím minimálního počtu signálových vodičů.

Vstup z tlačítek je veden přes posuvný registr CD4021, který realizuje paralelní vstup a sériový výstup (posuvný registr typu PISO – *Parallel-In Serial-Out*). Ten je řízen třemi signály:

- **STROBE** – aktivní hrana způsobí paralelní načtení okamžitých stavů všech vstupů do interního registru,
- **CLK** – hodinový signál synchronizující postupné vysouvání dat na sériový výstup,
- **OUT** – sériový datový výstup.

Protokol přenosu je striktně sekvenční: puls signálu STROBE uzamkne aktuální stav vstupů, načež každá náběžná hrana CLK vysune nejstarší uložený bit na výstup OUT. Stav osmi tlačítek je tak přenesen v osmi hodinových cyklech, aniž by bylo nutné rozšiřovat sběrnici o další datové vodiče.

Každý spínač je zapojen v konfiguraci *active-low* – jeho stisknutí je považováno za logickou 0 (napětí blízké referenčními GND).

Potlačení zákmitů tlačítek je řešeno skutečností, že čtení vstupu probíhá striktně před začátkem emulace snímku. Je pravděpodobný, že v okamžik čtení stavu tlačítek zákmit už není přítomen. V nejhorším případě, když čtení proběhne v přítomnosti zákmitu, může to způsobit zpoždění vstupu, ale v praxi je to zanedbatelný. Navíc je zákmit potlačen přirozenou délkou doby snímání dané hodinovou frekvencí CLK.

Kapitola 9

Návrh mechanických krytů

Součástí návrhu bylo také vytvoření mechanických krytů pro základní desku a ovladač. Modelování jednotlivých dílů bylo realizováno v CAD prostředí Fusion 360 s důrazem na funkčnost, jednoduchou montáž a dostatečnou mechanickou pevnost.

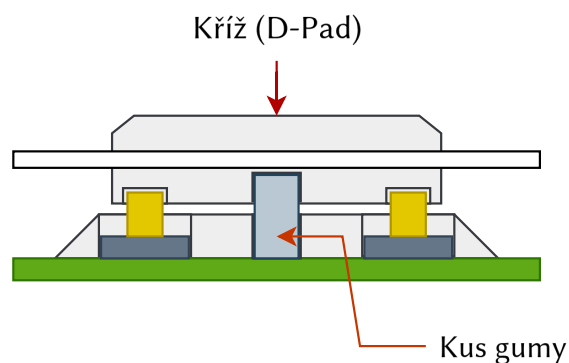
9.1 Kryt základní desky

Kryt základní desky je navržen jako dvoudílný, skládající se ze spodní části a horního víka. Spodní část slouží jako nosná struktura pro desku plošných spojů, která je v ní uložena a fixována pomocí montážních sloupků. Tyto sloupky zároveň slouží pro šroubové spojení s horním víkem. Součástí víka jsou také výztužné prvky, které zvyšují jeho tuhost a odolnost proti deformaci. Při návrhu byl kladen důraz na dostatečné izolační vzdálenosti a bezpečné oddělení vodivých částí.

9.2 Kryt ovladače

Kryt ovladače je rovněž řešen jako dvoudílný, tvořený spodní částí a víkem. Obě části jsou spojeny pomocí šroubů vedených skrze vnitřní sloupky. V horní části jsou integrovány ovládací prvky, konkrétně tlačítka a směrový kříž. Mechanická konstrukce těchto prvků je navržena tak, aby umožňovala přesné a spolehlivé ovládání.

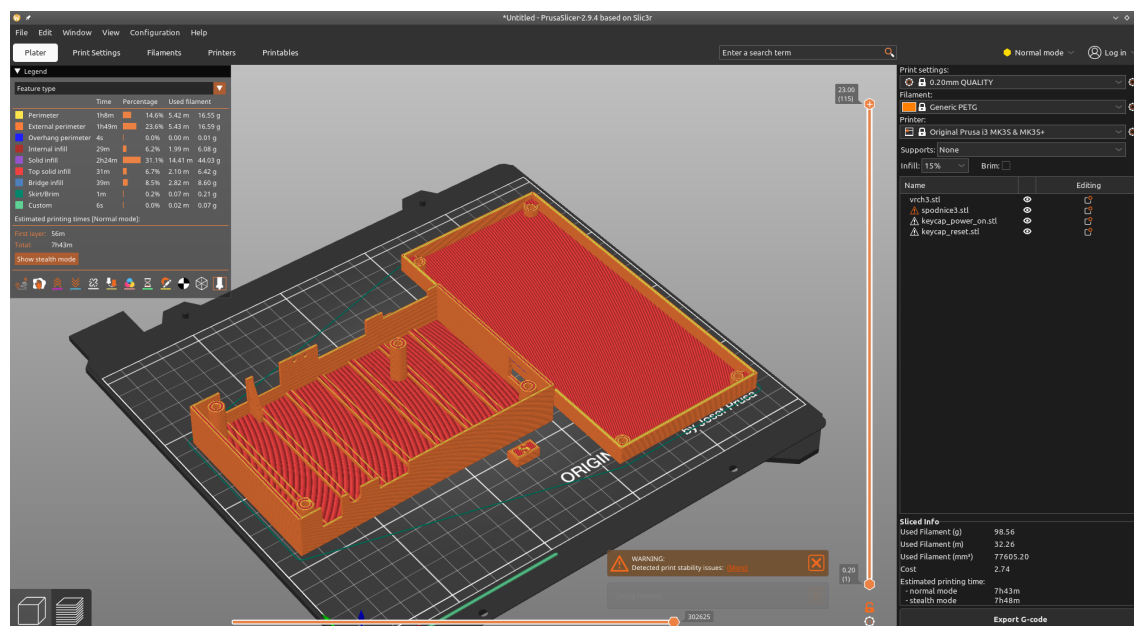
Směrový kříž obsahuje jednoduchý mechanický systém, který zabraňuje současnému stisku protilehlých směrů (např. vlevo a vpravo nebo nahoru a dolů), přičemž současně umožňuje kombinované (diagonální) stisky. Toho bylo docíleno použitím gumového válce (např. kus izolace kabelu), který brání kolmému stisknutí, ale díky své pružnosti dovoluje naklánět kříž (obr. 9.1). Nevýhodou toho je, že při dostatečně velkém stisku je stále možné stisknout současně tlačítka, ale za normálních podmínek tohle by nemělo stát. Možným řešením mohlo by být použití kovové nebo plastové vložky uvnitř gumy, která musí mít takový tvar, aby kříž se mohl naklánět, ale kolmé silové působení na gumu bylo nemožné.



Obrázek 9.1: Koncept kříže na ovladači.

9.3 Výroba

Navržené díly byly vyrobeny pomocí technologie 3D tisku na tiskárně Prusa i3 MK2S. Pro tisk byl použit filament PET-G, který byl zvolen s ohledem na jeho mechanickou pevnost, tepelnou odolnost a relativně snadnou zpracovatelnost. 3D tisk umožňuje rychlé ověření návrhu, případné úpravy a výrobu funkčních prototypů i hotových návrhů bez nutnosti použití složitějších výrobních technologií.



Obrázek 9.2: Příprava krytu základní desky pro 3D tisk v programu PrusaSlicer.

Kapitola 10

Návrh firmwaru

Firmware představuje softwarovou vrstvu, která propojuje jádro emulátoru s reálným hardwarem zařízení. Stejně jako samotný emulátor, firmware byl realizován v jazyce C, a to s využitím vývojové sady pico-sdk.

Hlavním úkolem firmwaru je správa běhu emulátoru, zajištění plynulého video a audio výstupu a obsluha vstupních herních ovladačů. Dále implementuje podporu souborového systému FAT pro čtení a zápis na SD kartu, což umožňuje práci se soubory jak samotnému firmwaru, tak emulovanému systému (prostřednictvím rozhraní FWX).

```
while (true) {
    /* Přečtení vstupu */
    update_joypads(&self->bus.joypad1, &self->bus.joypad2);
    /* Reset přerušení se nemá stát v okamžik zpracování
     * požadavku o resetování */
    disable_interrupts();
    /* Zpracování požadavku na obnovení výchozího stavu emulátoru */
    if (self->reset_required) {
        BusEvent new_ev = {
            .id = BUS_EVENT_RESET,
        };
        bus_add_event(&self->bus, &new_ev);
        self->reset_required = false;
    }

    enable_interrupts();
    /* Zpracování čekajících událostí na sběrnici */
    bus_update(&self->bus);
    /* Krok emulační smyčky (vykreslení jednoho snímku) */
    run_frame(self);
}
```

Výpis 13: Hlavní smyčka programu firmwaru.

Ve výpisu 13 je uveden kód hlavní smyčky firmwaru. V každé iteraci mikrokontrolér přečte aktuální stav ovladačů a provede emulaci jednoho úplného snímku obrazu. Před samotnou emulací snímku jsou zpracovány veškeré události nashromážděné od předchozího cyklu (například požadavek na načtení nového programu z SD karty). Běh emulace je primárně synchronizován s tokem audio signálu (viz podkapitola 10.2), což zajišťuje stabilní rychlost 60 snímků za sekundu.

Stisknutí resetovacího tlačítka na základní desce je obsluhováno asynchronně pomocí hardwareového přerušení. Toto přerušení pouze nastaví příznak `reset_required` na hodnotu `true`. Díky tomuto přístupu nemusí hlavní program v každém cyklu aktivně číst stav pinu (tzv. *polling*), čímž se šetří cenné procesorové instrukce a vymezuje situaci, když stisknutí nebylo zaregistrováno.

Z rychlostních důvodů, RP2350 je přetaktován (*overclocked*) z původních 150 MHz na 300 MHz.

10.1 Generace video výstupu DVI

Pro generování DVI signálu je využívána knihovna PicoDVI, která zajišťuje přesné časování, synchronizaci a samotné generování TMDS signálů potřebných pro přenos obrazu na HDMI.

Výsledný obraz je emulován v rozlišení 320×240 pixelů. Pro reprezentaci barev byl zvolen formát RGB565, kde každý pixel zabírá v paměti přesně 16 bitů (2 byty). Tento formát byl upřednostněn před standardním formátem RGB888 (24 bitů na pixel) z důvodu úspory paměti RAM a propustnosti sběrnice. Vzhledem k tomu, že původní barevná paleta konzole NES je poměrně omezená, redukce barevné hloubky nepřináší žádnou znatelnou ztrátu vizuální kvality.

DVI je inicializován ve standardním režimu VGA, který fyzicky generuje rozlišení 640×480 pixelů při obnovovací frekvenci 60 Hz (viz výpis 14). Dle standardu je tento režim označován jako *Low Pixel Format* s taktovací frekvencí pixelových hodin 25,175 MHz, což představuje zátěžově nejméně náročný režim podporovaný moderními monitory[15]. Vzhledem k tomu, že interní rozlišení NESu je pouze 256×240 pixelů, je tento režim pro potřeby emulátoru dostačující.

```
#include <dvi.h>
#include <dvi_serialiser.h>

static struct dvi_inst g_dvi;

void core1_main() {
    /* Knihovna PicoDVI přebírá kontrolu nad druhým jádrem
     * procesoru (Core 1), které je dedikováno výhradně pro
     * přesné časování a generování DVI signálu. */
    dvi_register_irqs_this_core(&g_dvi, DMA_IRQ_0);
    dvi_start(&g_dvi);
    dvi_scanbuf_main_16bpp(&g_dvi);
    /* dvi_scanbuf_main_16bpp() běží v nekonečné smyčce
     * a zajišťuje generování obrazu */
    __builtin_unreachable();
}

void init_dvi(void) {
    /* Konfigurace VGA režimu (640x480 pixelů @ 60 Hz) */
    g_dvi.timing = &dvi_timing_640x480p_60hz;
    g_dvi.ser_cfg = DVI_DEFAULT_SERIAL_CONFIG;
    /* Registrace callback funkce pro přípravu nového řádku obrazu */
    g_dvi.scanline_callback = prepare_dvi_scanline;

    dvi_init(
        &g_dvi, next_stripped_spin_lock_num(), next_stripped_spin_lock_num()
    );
    /* Registrace callback funkce pro přípravu nového řádku obrazu */
    multicore_launch_core1(core1_main);
}
```

Výpis 14: Inicializace DVI.

Pro stabilitu video obrazu se udržuje framebuffer s rozlišením 256x240, kde každý pixel je reprezentován indexem do určité palety. Tento přístup je více úsporný z hlediska využití paměti, než udržovat kompletní framebuffer s již dekodovanými pixely.

Knihovna PicoDVI dovoluje nastavení callback funkce, která je automaticky volána vždy, když je videovýstup připraven na odeslání dalšího řádku. Funkce `prepare_dvi_scanline()` načte z bufferu odpovídající řádek bytů, pomocí LUT tabulky převede indexy na finální 16bitové barvy a výsledek předá do fronty řádků PicoDVI. Tento přístup šetří RAM paměť, jelikož skutečné 16bitové barvy se konstruuji dynamicky pouze pro jeden právě vykreslovaný řádek.

Jelikož fyzické DVI rozlišení má šířku 320 pixelů, ale obraz generovaný systémem NES je široký pouze 256 pixelů, je nutné provést horizontální centrování. Toho se dosahuje vložení symetrických černých pruhů z levé i pravé strany (tzv. *pillarbox*), což zabraňuje nechtěnému roztažení obrazu a zachovává původní poměr stran (*aspect ratio*). V našem případě pro šetření procesorového času při dekódování pixelů jsme využili toho, že statické proměnné v C jsou vynulovány, a dekódované pixely vykreslujeme jen do vyhrazené oblasti v bufferu řádku, proto není potřeba opakovaně centrovat.

Jak již bylo zmíněno v podkapitole 6.4, PPU signalizuje dokončení vykreslení jednoho řádku pomocí definovaného *callbacku*. Zaregistrovaná obslužná funkce `prepare_ppu_scanline()` ihned přepokopíruje data z interního bufferu PPU přímo do příslušného řádku hlavního framebufferu.

10.2 Generace audio výstupu

V podkapitole 7.3 bylo zmíněno, že pro generaci audio signálu z výstupu APU je využívána pulsně šířková modulace (PWM) pro aproximaci audio signálu v digitální doméně. Naše řešení kromě PWM také využívá technologii DMA pro synchronní a stabilní přenos vzorku podle vzorkovací frekvence.

10.2.1 Konfigurace PWM

Pro dosažení kvalitního a stabilního zvukového výstupu je nezbytné zajistit, aby byly jednotlivé vzorky odesílány do PWM generátoru přesně v rytmu vzorkovací frekvence f_s . Jakákoliv softwarová nepravidelnost (tzv. *jitter*) by se projevila jako slyšitelné zkreslení (trhaní zvuku, zasekávání atd.). Z tohoto důvodu byla zvolena plně hardwarová cesta využívající řetězení kanálů DMA, díky čemuž procesor věnuje tomu minimální svůj čas a je menší šance desynchronizace.

Nejprve je nutné odvodit parametry PWM z frekvence systémových hodin f_{clk} . Pro dosažení optimální filtrace v analogové části obvodu byla zvolena nosná frekvence PWM $f_{PWM} = 250 \text{ kHz}$. Vzorkovací frekvence emulátoru je pevně dána na $f_s = 31,25 \text{ kHz}$. Z toho vyplývá, že na každý jeden audio vzorek připadá přesně 8 celých PWM cyklů:

$$\frac{f_{PWM}}{f_s} = \frac{250000}{31250} = 8$$

Nosná Frekvence PWM byla zvolena jako celočíselný násobek vzorkovací frekvence, aby bylo zajištěno deterministické a periodické časování mezi aktualizací střídy PWM a jednotlivými vzorky audio signálu. Tím je zaručeno, že každý vzorek je reprezentován konstantním počtem period PWM, což eliminuje časovou nekonzistenci při přepisu hodnot (*jitter*) a zabraňuje vzniku nízkofrekvenčních artefaktů způsobených nesynchronním vzorkováním. V případě neceločíselného poměru by docházelo k postupnému fázovému driftu mezi PWM signálem a generováním

vzorků, což by vedlo k periodickému kolísání efektivní šířky pulzu a degradaci kvality rekonstruovaného analogového signálu.

Klíčovým parametrem PWM je hodnota stropu čítače (TOP), která definuje maximální rozlišení (bitovou hloubku) výstupního signálu, závislé na počtu možných stavů stropu čítače TOP:

$$\text{bitová hloubka} = \log_2(\text{TOP} + 1)$$

Vztah pro výpočet frekvence PWM v mikrokontroléru RP2350 je definován jako:

$$f_{\text{PWM}} = \frac{f_{\text{clk}}}{\text{clkdiv} \cdot (\text{TOP} + 1)}$$

Jelikož nosní frekvenci PWM f_{PWM} máme vybranou a taktovací frekvenci RP2350 f_{clk} známe, potřebujeme vypočítat optimální hodnotu pro strop čítače TOP. Po přeskládání rovnice a dosazení hodnot získáváme:

$$\text{TOP} = \frac{f_{\text{clk}}}{f_{\text{PWM}}} - 1 = \frac{300 \cdot 10^6}{250 \cdot 10^3} - 1 = 1200 - 1 = 1199$$

Rozlišení PWM výstupu je tedy 1200 úrovní (včetně 0). Bitovou hloubku získáme hodnotu přibližně 10,2 bitů ($\log_2(1200) \approx 10,2$). Ačkoliv moderní audio systémy využívají 16bitové nebo 24bitové rozlišení, pro reprodukci 8bitového zvuku z konzole NES je 10bitová hloubka více než dostatečná.

Jelikož APU generuje 16bitové vzorky (v rozsahu 0 až 65535), je nutné je před odesláním do komparátoru přemapovat na rozsah čítače (0 až 1199):

$$\text{hodnota} = \left\lfloor \frac{\text{vzorek} \cdot \text{TOP}}{65535} \right\rfloor$$

Funkce `init_pwm()` ve výpisu 15 provádí konfiguraci a inicializaci PWM podle výše uvedených informací.

10.2.2 Architektura DMA přenosu

Aby byl zajištěn nepřerušovaný tok dat, je využit mechanismus dvojitého bufferování, známý také jako **ping-pong buffer** [17]. Paměťový prostor pro vzorky je rozdělen do dvou polí. Zatímco emulátor plní novými daty jeden buffer, DMA řadič čte data z druhého bufferu a posílá je do PWM komparátoru.

```

/* Konvertace vzorku na hodnotu čítače. Přetypování na uint32_t
 * je zapotřebí, pokud výsledek násobení bude větší než 65535. */
#define PWM_VALUE(smp) \
    ((uint16_t) (((uint32_t) (smp) * g_pwm_wrap) / 65535)))

#define PWM_FREQ 250000

static unsigned g_pwm_slice; /* číslo slicu */
static uint16_t g_pwm_wrap; /* TOP */

void init_pwm(void) {
    /* Nastavení pinu jako výstup PWM */
    gpio_set_function(APU_OUT_PIN, GPIO_FUNC_PWM);

    g_pwm_slice = pwm_gpio_to_slice_num(APU_OUT_PIN);

    /* Výpočet TOP (wrap v terminologii RP2350) */
    unsigned sys_clk = clock_get_hz(clk_sys);
    g_pwm_wrap = sys_clk / PWM_FREQ - 1;
    /* Inicializace PWM */
    pwm_set_wrap(g_pwm_slice, g_pwm_wrap);
    pwm_set_clkdiv(g_pwm_slice, 1.f); /* dělička je 1 */
    pwm_set_enabled(g_pwm_slice, true);
}

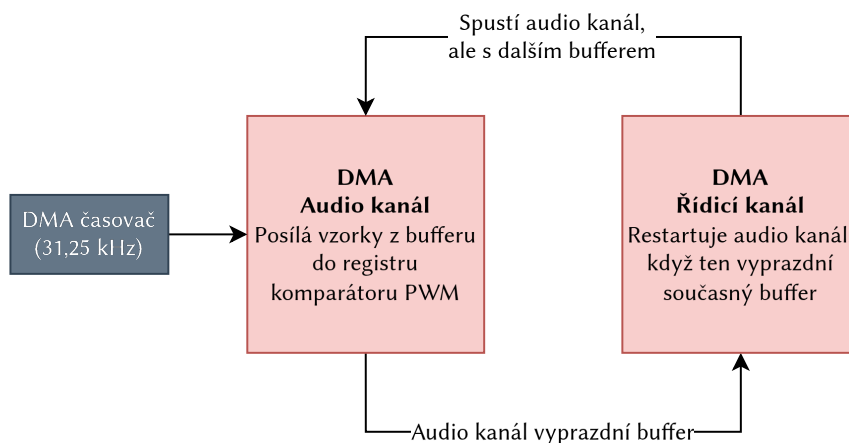
```

Výpis 15: Inicializace a konfigurace PWM.

K řízení tohoto procesu jsou zapotřebí dva DMA kanály (obr. 10.1):

1. **Audio kanál** – tento kanál je zodpovědný za přenos samotných vzorků (16bitových hodnot) z aktuálního bufferu přímo do hardwarového registru PWM komparátoru. Přenos jednoho vzorku je spouštěn speciálním DMA časovačem, který je nastaven přesně na vzorkovací frekvenci $f_s = 31,25$ kHz.
2. **Řídicí kanál** – jakmile audio kanál vyprázdní aktuální buffer, automaticky spustí tento řídicí kanál. Jeho jediným úkolem je přepsat zdrojovou adresu audio kanálu na adresu druhého bufferu. Následně řídicí kanál opětovně spustí audio kanál, čímž se cyklus uzavírá do nekonečné smyčky.

Tento řetězený přístup zaručuje, že nedochází k žádnému softwarovému zpoždění při přepínání bufferů. Alternativním řešením by bylo zaregistrovat přerušení, které by se vyvolalo když jediný DMA audio kanál dokončí přenos dat. Ve vývoji se ale ukázalo, že to naopak způsobuje zpomalení procesoru, čímž je ovlivněna plynulost běhu emulaci, a také vyvolává trhání zvuku.



Obrázek 10.1: Blokové schéma principu dvou DMA kanálů: audio kanál a řídicí kanál.

10.2.3 Konfigurace DMA

Teoretický koncept ping-pong bufferu a řetězení kanálů je v realizován pomocí sady funkcí pico-sdk.

Při inicializaci DMA, je nutné nastavit frekvenci DMA časovače f_{DMA} na potřebnou. Obecný vztah pro f_{DMA} platí:

$$f_{\text{DMA}} = f_{\text{clk}} \frac{n}{d}$$

V našem případě chceme docílit toho, aby frekvence časovače DMA se rovnala vzorkovací frekvenci, takže potřebujeme se zbavit členu f_{clk} z rovnice. Když dosadíme $n = f_s$ a $d = f_{\text{DMA}}$, dostaneme:

$$f_{\text{DMA}} = f_{\text{clk}} \frac{n}{d} = 300 \cdot 10^6 \cdot \frac{3,125 \cdot 10^4}{300 \cdot 10^6} = 31,25 \text{ kHz}$$

Knihovna pico-sdk poskytuje funkci `dma_timer_set_fraction(timer, n, d)` pro nastavení frekvence DMA časovače. Jelikož je ale taktovací frekvence jádra 300 MHz, tato hodnota přesahuje maximální limit 16bitového dělitele. Problém byl vyřešen zkrácením zlomku na poměr $\frac{1}{9600}$, což přesně odpovídá požadovaným 31,25 kHz.

10.2.4 Generace vzorků

Zápis vzorků do správného bufferu zajišťuje funkce `audio_queue()` (výpis 16). Pokaždé když APU vygeneruje nový vzorek, je tato funkce zavolána. Jakmile dojde k naplnění aktuálního bufferu, musí se přejít na další buffer.

Tato funkce také zajišťuje synchronizaci v okamžiku, když CPU zaplní buffer a chce přepnout se na další: CPU zkontroluje hardwarový registr `read_addr` DMA řadiče. Pokud DMA stále čte z druhého bufferu, znamená to, že CPU emuluje systém rychleji, než probíhá reálné přehrávání zvuku. Procesor se proto zablokuje ve smyčce `tight_loop_counter()`, dokud se buffer neuvolní. Tímto způsobem emulátor se také synchronizuje se zvukem a udržuje stabilní rychlost běhu bez nutnosti využití softwarových časovačů.

10.3 Obsluha SD karty

Pro ukládání a načítání obrazů her (ROM) byl systém vybaven rozhraním pro SD karty. Komunikace probíhá prostřednictvím rozhraní SPI mikrokontroléru RP2350. Aby byla zajištěna modularita kódu a snadná správa souborů, je softwarové řešení rozděleno do tří abstraktních vrstev:


```

void audio_queue(ApuSample smp) {
    /* Přemapování 16bitového vzorku na rozlišení PWM a zápis do bufferu */
    g_bufs[g_curr_buf_idx][g_buf_pos++] = PWM_VALUE(smp);

    /* Pokud je aktuální buffer plný, připravíme se na přepnutí */
    if (g_buf_pos >= AUDIO_BUFFER_SIZE) {
        unsigned next_buf_idx = (g_curr_buf_idx + 1) & (RING_SIZE - 1);

        for (;;) {
            /* Získání aktuální adresy, ze které DMA právě čte */
            uintptr_t dma_addr = dma_hw->ch[g_audio_chan].read_addr;
            uintptr_t next_buf_start = (uintptr_t) g_bufs[next_buf_idx];
            uintptr_t next_buf_end = next_buf_start + sizeof(g_bufs[0]);

            /* Pokud DMA už z dalšího bufferu nečte, můžeme smyčku opustit */
            if (dma_addr < next_buf_start || dma_addr >= next_buf_end)
                break;

            /* Emulátor běží příliš rychle, CPU musí počkat na hardware audia */
            tight_loop_contents();
        }

        /* Přepnutí indexů pro další dávku vzorků */
        g_curr_buf_idx = next_buf_idx;
        g_buf_pos = 0;
    }
}

```

Výpis 16: Ukládání vzorku do fronty (volného bufferu).

1. **Hardwarová vrstva (Platform layer)** zajišťuje nízkourovňovou inicializaci SPI řadiče a GPIO pinů mikrokontroléru RP2350. Tato část je jedinou platformě závislou vrstvou, která přímo manipuluje s registry procesoru.
2. **Ovladač SD karty (Storage layer)** implementuje komunikační protokol SD karet v režimu SPI. Zajišťuje inicializační sekvenci média, identifikaci typu karty (SDSC/SDHC) a především blokové čtení a zápis dat (standardizované sektory o velikosti 512 bytů).
3. **I/O rozhraní pro FatFs** slouží jako propojovací most (wrapper) mezi ovladačem karty a knihovnou souborového systému.

Pro efektivní práci se soubory a kompatibilitu s osobními počítači je zapotřebí implementovat souborový systém. Na to byl zvolen souborový systém FAT32, který je standardem pro vystavené systémy, a byla zvolena knihovna FatFs, která realizuje tento souborový systém.

Díky této vrstvené architektuře se SD karta v aplikaci jeví jako standardní disková jednotka. Emulátor tak může využívat běžné funkce pro práci se soubory (otevření souboru, sekvenční čtení), což značně usnadňuje implementaci zavaděče ROM. Podrobná dokumentace inicializačních příkazů a stavových diagramů SD karty je pro zájemce uvedena v příloze F.

10.4 Čtení vstupu z ovladačů

Pro zajištění přesného časování řídicích signálů a hardwarové akcelerace komunikace byl využit subsystém PIO (*Programmable I/O*), který je specifický pro mikrokontroléry RP2040/RP2350.

10.4.1 Implementace

Inicializace PIO automatu probíhá pomocí dedikovaných funkcí:

- `pio_claim_free_sm_and_add_program()` – zkompileovaný PIO program nahraje do instrukční paměti, zkonfiguruje mapování příslušných GPIO pinů a nastaví děličku hodin. Druhá funkce nastaví piny.
- `joypad_read_program_init()` – nastaví piny na potřebné vývody.

Komunikace je rozdělena na hardwarovou část (PIO program) a softwarovou obsluhu v jazyce C. PIO program (výpis 17) je navržen tak, aby po obdržení signálu od procesoru vygeneroval impuls na pinu LATCH a následně pomocí osmi hodinových cyklů na pinu CLOCK paralelně přečetl datové bity z obou posuvných registrů.

Taktování PIO automatu je softwarově sníženo na frekvenci přibližně 1 MHz (souhlasí s data-sheetem posuvného registru CD4021), takže spuštění jedné instrukce trvá 1 μ s.

```
; PIO program pro taktování čipu CD4021 a čtení jeho sériového výstupu
.program joypad_read

; Set piny ovládají signál LATCH (Strobe)
; Sideset piny ovládají signál CLOCK
; In piny čtou datové výstupy ovladačů
.side_set 1

; Definice zpoždění (v cyklech) pro dodržení časování CD4021 (~15 us)
.define CLK_TIME 15
.define STR_TIME 15

.wrap_target
    pull block            side 0            ; Čekání na příkaz od CPU (přes TX FIFO)
    mov isr, null         side 0

    set pins, 1           side 0            ; LATCH = 1 (Vzorkování tlačítek)
    nop                   side 0 [STR_TIME] ; Prodleva
    set pins, 0           side 0            ; LATCH = 0

    set x, 7              side 0            ; Nastavení počítadla smyčky na 8 iterací
read_bit:
    in pins, 2            side 0 [CLK_TIME] ; CLK = 0, přečtení 2 pinů současně
    jmp x-- read_bit      side 1            ; CLK = 1, skok na další bit

    push block           side 0            ; Odeslání nasbíraných dat do RX FIFO
.wrap
```

Výpis 17: Zdrojový kód PIO programu pro asynchronní čtení ovladačů.

Před zahájením emulace každého snímku je volána funkce `update_joypads()`. Její obsluha spočívá v odeslání prázdného povelu (nuly) do TX fronty stavového automatu, čímž se spustí smyčka PIO programu. PIO následně přečte 8 dvojic bitů (jeden bit pro každý ovladač v každém cyklu) a 16bitový výsledek vrátí přes RX frontu. Datové slovo je následně v jazyce C bitovými operacemi rozděleno do samostatných bytů pro první a druhý ovladač. Tato funkce také zajišťuje synchronizaci s PIO blokem.

Kapitola 11

Testování a ověření správnosti

Testování probíhalo v několika úrovních – od izolovaných unit testů procesoru a testovacích programů až po porovnání s referenčními emulátory.

11.1 Testování CPU

Při implementaci emulátoru CPU byla vytvořena sada unit testů pomocí knihovny Unity. Tyto testy sloužily jako prvotní ověření správnosti implementace a zaměřovaly se na funkčnost jednotlivých instrukcí a jejich cyklovou přesnost.

Pro důkladnější testování byly využity veřejně dostupné testovací programy:

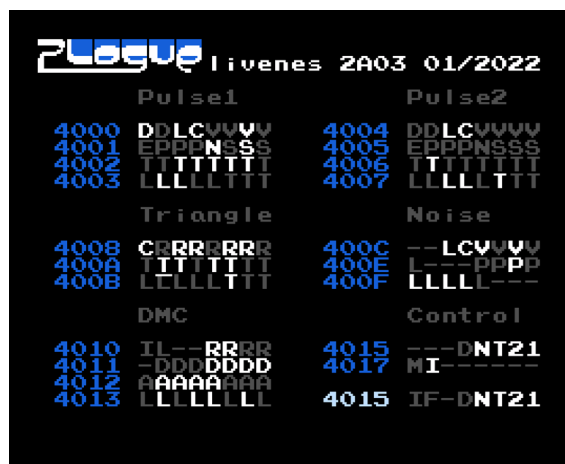
- **Nestest** – tento testovací program generuje log o stavu všech registrů a počtu uplynulých cyklů po každé vykonané instrukci. Výstup emulátoru byl automatizovaně porovnáván s referenčním logem (*golden log*), což umožnilo přesně identifikovat případné odchylky v chování a časování CPU.
- **Blargg's¹ Instruction Tests (instr_test-v5)** – tato sada ROM souborů testuje pokročilé aspekty CPU, včetně neoficiálních instrukcí. Emulátor úspěšně prošel všemi testy s výjimkou neoficiálních instrukcí SHA, SHX, SHY a TAS. Tyto instrukce závisejí na aktuálním stavu a šumu na datové sběrnici, proto jejich emulace je celkem obtížná. Navíc tyto instrukce se téměř nepoužívají, proto absence jejich emulace neuškodí kompatibilitě emulátoru.

11.2 Komplexní testování

Na rozdíl od CPU, PPU a APU se netestovaly izolovaně pomocí unit testů, nýbrž prostřednictvím porovnání s referenčním emulátorem **Mesen**, který je považován za jeden z nepřesnějších emulátorů NES[18], a předmětem testování byly komerční hry a testovací programy. Komerční hry byly využité hlavně na testování PPU a na ověření, zda jednotlivé emulované komponenty spolupracují správně.

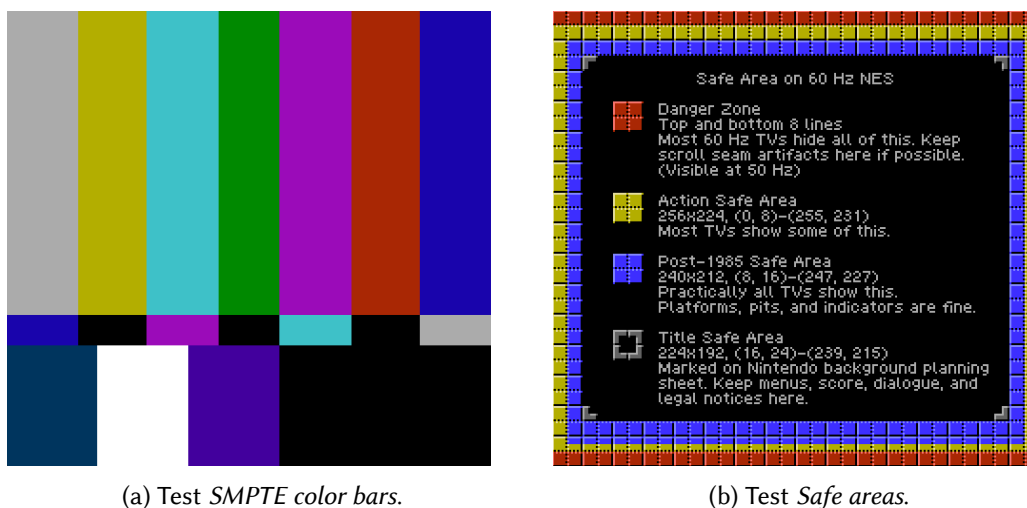
¹Autorem je uživatel fóra NESDev s přezdívkou „Blargg“, který velice aktivně se věnoval zkoumání architektury NES a návrhu testovacích programů pro emulátory.

Komerční hry také dovolovaly testovat správnost emulace APU, ale jelikož většina her nevyužívá všechny možnosti APU, správnost se dalo ocenit jen hrubě. Pro detailnější testování byl využit program **livenes**, který funguje jako syntezátor, kde uživatel může přepínat jednotlivé bity registrů APU (obr. 11.1). Díky tomuto programu bylo možné tence otestovat jednotlivé aspekty APU.



Obrázek 11.1: Program livenes, který dovoluje přepínat jednotlivé bity registrů APU.

V případě PPU jedním z užitečných programů pro ověření správnosti grafického výstupu byl program **240p-test-mini**. Tento program poskytuje rozsáhlou sadu testů pro ověření správnosti grafického výstupu, přičemž nejen emulátoru, ale také monitoru nebo televizoru (obr. 11.2a a 11.2b).



(a) Test SMPTE color bars.

(b) Test Safe areas.

Obrázek 11.2: Testovací sada 240p-test-mini.

Pro hlubší verifikaci a ověření správnosti emulace na úrovni nejjemnějších detailů se v další fázi vývoje plánuje nasazení rozsáhlé sady specializovaných programů a zátěžových testů obsažených v archivu [19]. Tyto testy jsou navrženy k odhalení specifických hardwarových vlastností, které jsou pro platformu NES typické, avšak v softwarových implementacích bývají často opo-

míjeny nebo chybně implementovány. Pozornost bude věnována zejména kritickým aspektům časování instrukcí a reakce CPU na přerušení, komplexnímu chování PPU během vykreslování a precizní syntéze v rámci APU.

11.3 Benchmarking a profilování

Základní metrika výkonu byla získána měřením času potřebného pro kompletní vygenerování jednoho snímku. K tomuto účelu byly využity hardwarové časovače poskytované knihovnou Pico SDK, přesněji funkce `time_us_32()` (výpis 18).

```
while (true) {
    unsigned t_start = time_us_32();

    update_joy pads(&self->bus.joypad1, &self->bus.joypad2);
    disable_interrupts();

    if (self->reset_required) {
        /* ... */
    }

    enable_interrupts();
    bus_update(&self->bus);
    run_frame(self);

    unsigned t_end = time_us_32();
    /* Uložení rozdílu času před a po emulaci snímku */
    frame_times[frame_time_idx] = t_end - t_start;
    /* Výpis naměřených hodnot a jejich průměr */
    if (frame_time_count >= FRAME_TIMES_TOTAL) {
        frame_time_idx = 0;
        print_average_frame_time();
    }
}
```

Výpis 18: Měření průměrného času, potřebného na emulaci jednoho snímku.

Měření probíhala přímo v hlavní emulační smyčce. Před zahájením emulace každého snímku byla sejmuta počáteční hodnota systémového časovače v mikrosekundách a po dokončení kompletního vykreslovacího cyklu PPU (zahrnujícího i neviditelné řádky) byla zaznamenána hodnota koncová. Rozdíl těchto dvou stavů definuje čistý výpočetní čas nezbytný pro zpracování jednoho snímku. Pro eliminaci výkyvů a získání statisticky relevantních dat byl implementován buffer o velikosti N prvků. Do tohoto pole se naměřené hodnoty postupně ukládaly a při každém jeho úplném naplnění byl vypočítán a do konzole vypsán aritmetický průměr všech časů.

Při konfiguraci $N = 256$ se ukázalo, že průměrný čas potřebný na emulaci jednoho snímku se stabilně pohybuje kolem hranice 16,667 ms (což přesně odpovídá frekvenci 60 snímků za sekundu), a to jak v rozhraní BIOSu, tak i v průběhu samotného hraní většiny her. Tato časová hodnota mírně narůstala pouze při nadprůměrném zatížení emulačního jádra – například při vykreslování maximálního počtu spritů nebo při intenzivní manipulaci s registry PPU a APU. Naměřený rozdíl však zůstával minimální a z hlediska plynulosti emulace prakticky nepostřehnutelný.

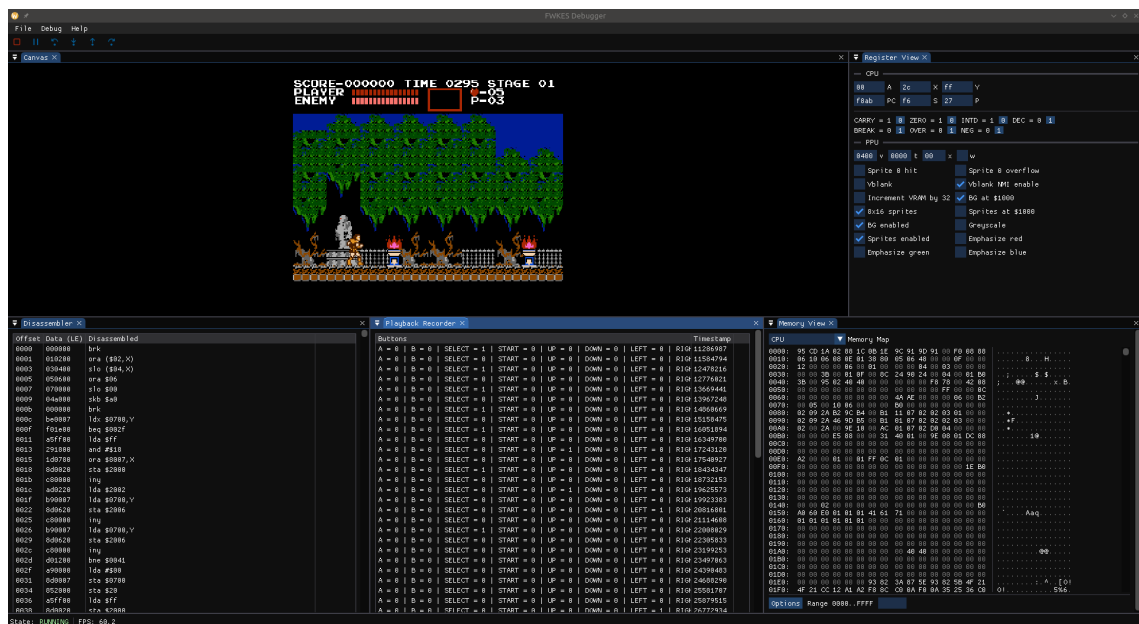
Měření pomocí softwarových časovačů a testovacích ROM obrazů spolehlivě prokazuje schopnost emulátoru běžet v reálném čase, ale přispívá k odchylce, jelikož se přidává režie pro záznam měřených hodnot a jejich výpis do konzole. Pro přesnější hodnoty se plánuje použít hardwarové

profilování pomocí GPIO — přesnějšího měření by bylo možné dosáhnout vyhrazením jednoho GPIO pinu mikrokontroléru, jehož stav by se měnil na začátku a na konci emulaci snímku. Analýzou tohoto signálu pomocí osciloskopu nebo logického analyzátoru by bylo možné exaktně měřit výpočetní čas a snímkovou frekvenci s téměř nulovou softwarovou režií.

11.4 Vlastní ladicí nástroje

S růstem komplexity projektu při emulaci PPU a APU, paralelně s emulátorem se pracovalo na vlastním ladicím nástroji (debuggeru), který umožňuje komplexní testování nejen emulátoru, ale i emulovaných programů.

Tento nástroj je podobný běžným debuggerům, které se dá najít ve většině IDE: nabízí funkce jako zobrazování a modifikace registrů všech emulovaných komponentů, pohled na paměť v hex editoru, nastavení breakpointů apod. Tento nástroj umožnil detailní ladění emulátoru na desktopu, jelikož poskytoval pohodlné prostředky pro zobrazení stavu, ovládání a konfiguraci emulátoru. Kromě rozšíření ladicích možností, to také v dlouhodobém pohledu ušetřilo čas a energii.



Obrázek 11.3: Screenshot debuggeru.

Do budoucna se plánuje pokročit dál a vyvinout plnohodnotné vývojové prostředí, které dovolí nejen ladit NES programy, ale také i psát vlastní.

11.5 Závěr

Výsledný emulátor dosahuje vysoké míry věrnosti původnímu hardwaru, která v mnoha ohledech překonává běžné amatérské implementace. Přestože z hlediska absolutní přesnosti zatím nedosahuje úrovně dlouhodobě vyvíjených a vysoce precizních projektů, jako jsou Mesen, FCEUX nebo Nintendulator, je schopen korektně a plynule spustit drtivou většinu titulů využívajících podporované mappery (NROM, UNROM a MMC3). V těchto případech obrazový i zvukový výstup věrně odpovídá chování originálního systému.

Drobné odchylky se projevují pouze ve velmi specifických scénářích, nejčastěji v podobě drobných vizuálních artefaktů. Ty jsou důsledkem vědomého zjednodušení některých pokročilých vlastností PPU, konkrétně absence podpory pro emphasis bity a základní implementace ořezu okrajů obrazu (*clipping*). Vzhledem k tomu, že tyto specifické funkce využívá pouze úzká skupina her, zůstává celková hratelnost a uživatelský zážitek u naprosté většiny testovaných titulů neovlivněn.

Kapitola 12

Závěr

Předložená práce se zabývá návrhem a realizací emulační platformy herní konzole Nintendo Entertainment System na mikrokontroléru RP2350. Cílem bylo ověřit, zda je možné implementovat komplexní historickou architekturu v prostředí s omezenými hardwarovými prostředky a bez využití operačního systému. Tento cíl byl splněn. Byla vytvořena funkční emulační platforma, která implementuje klíčové subsystémy původní konzole, konkrétně procesor Ricoh 2A03, grafickou jednotku PPU a zvukový subsystém APU, a je schopna vykonávat programy ve formě ROM obrazů a generovat odpovídající obrazový i zvukový výstup.

Ve srovnání s běžnými softwarovými emulátory běžícími na osobních počítačích se navržené řešení zásadně liší přístupem k řízení systému. Zatímco klasické emulátory využívají služby operačního systému a vyšší úrovně abstrakce, zde bylo nutné explicitně řešit časování, synchronizaci i komunikaci s periferiemi. Tento přístup se ukázal jako náročnější na návrh i implementaci, avšak zároveň umožňuje přesnější kontrolu nad chováním systému a lepší pochopení principů počítačové architektury.

Zásadní část práce představovala implementace jednotlivých subsystémů s důrazem na cyklovou přesnost. Ukázalo se, že právě synchronizace CPU, PPU a APU je jedním z nejnáročnějších problémů. Původní konzole využívá paralelně běžící specializované čipy, zatímco v navrženém řešení jsou všechny komponenty vykonávány sekvenčně. Použitý „catch-up“ mechanismus se ukázal jako funkční kompromis, který umožňuje zachovat časovou konzistenci systému i při sekvenčním zpracování na jednom mikrokontroléru, a tím zachovat správné chování programů závislých na přesném časování.

Výsledky práce zároveň ukazují limity tohoto přístupu. Omezený výpočetní výkon mikrokontroléru se projevuje například vizuálními artefakty nebo nutností optimalizací, které mohou být v rozporu s ideální přesností. Dalším omezením je neúplná implementace mapovacích radičů, což snižuje kompatibilitu s některými programy. Tyto nedostatky však nevyplývají z koncepce řešení, ale spíše z rozsahu implementace a dostupných prostředků, a představují přirozený prostor pro další vývoj.

Významným přínosem práce je návrh komunikační linky FWX, která rozšiřuje klasický model emulace o možnost přímé interakce mezi emulovaným programem a hostitelským systémem. Díky tomuto rozhraní může program přistupovat k externímu úložišti, konfigurovat chování

systému nebo komunikovat s periferiemi. Tento princip překračuje rámec tradiční emulace a posouvá zařízení směrem k obecné výpočetní platformě. Systém tak není omezen možnostmi původní konzole NES a má potenciál stát se libovolně konfigurovatelným embedded zařízením podle potřeb uživatele.

Do budoucna lze navrhnout zejména optimalizaci emulačního jádra, rozšíření podpory mapovacích řadičů a další vývoj komunikační linky FWX, například o přímé napojení na GPIO rozhraní. To by umožnilo ještě širší integraci s externími zařízeními a další posun směrem k univerzální embedded platformě.



Obrázek 12.1: Finální výrobek.

Je důležité poznamenat, že tento výrobek v aktuálním stavu není konkurenceschopný s takovými projekty jako např. RetroPie, který poskytuje vyspělou open-source emulační platformu s obrovskou komunitou. Soupeření však není naším cílem. Naším prvotním cílem bylo ukázat, že je možné vytvořit zcela přesný emulátor na malovýkonné platformě (tento případě mikrokontrolér RP2350) s využitím správných optimalizací a přístupů při implementaci. Dále jsme řešili problémy, jako je komunikace mezi emulátorem a emulačním programem pro integraci BIOSu, vytvoření jednoduchého, ale funkčního zvukového ovladače, výroba výsledného zařízení s integrací všech vrstev atd. Díky tomu, že se jedná o open-source projekt licencovaný pod GNU GPL v2 (softwarová část), tento projekt slouží jako referenční emulátor pro lidi, které se zabývají emulací retro systému, a to zejména pro malovýkonné platformy. Nicméně, při vývoji jsme se snažili udělat zařízení snadno vyrobitelné, a uživatelský přívětivé. V budoucnu plánujeme vylepšovat náš projekt ve všech aspektech: uživatelská přívětivost a pohodlnost, ergonomii ovladačů a mechanickou část, elektrickou část, komercializaci apod.

Použitá literatura

1. *High and low-level emulation* [online]. [cit. 2026-03-31]. Dostupné z: https://emulation.gametechniki.com/index.php/High_and_low-level_emulation.
2. BARRIO, Victor Moya del. Study of the techniques for emulation programming. 2001.
3. WIKIMEDIA COMMONS. *File:Pcm.svg* — *Wikimedia Commons, the free media repository* [online]. 2022. [cit. 2026-03-31]. Dostupné z: <https://commons.wikimedia.org/w/index.php?title=File:Pcm.svg&oldid=714630838>.
4. PARKER, Michael. *Digital signal processing 101: everything you need to know to get started*. Newnes, 2010. ISBN 978-1856179218.
5. *NESdev Wiki* [online]. [cit. 2026-03-31]. Dostupné z: https://www.nesdev.org/wiki/Nesdev_Wiki.
6. COPETTI, Rodrigo. *NES/Famicom Architecture / A Practical Analysis* [online]. [cit. 2026-03-31]. Dostupné z: <https://www.copetti.org/writings/consoles/nas/>.
7. *S6500 – 8-Bit Microprocessor Family*. Dostupné také z: <https://www.princeton.edu/~mae412/HANDOUTS/Datasheets/6502.pdf>.
8. STEIL, Michael. *How MOS 6502 Illegal Opcodes Really Work* [online]. 2008. [cit. 2026-03-31]. Dostupné z: <https://www.pagetable.com/?p=39>.
9. *NMOS 6510 Unintended Opcodes*. 2024. Dostupné také z: <https://csdb.dk/release/download.php?id=305221>.
10. MORLAN, Austin. *An Overview of NES Rendering - Austin Morlan* [online]. [cit. 2026-03-31]. Dostupné z: https://austinmorlan.com/posts/nas_rendering_overview/.
11. WIKIMEDIA COMMONS. *File:NES palette.png* — *Wikimedia Commons, the free media repository* [online]. 2024. [cit. 2026-03-31]. Dostupné z: https://commons.wikimedia.org/w/index.php?title=File:NES_palette.png&oldid=904542414.
12. *NES Palette Tricks* [online]. [cit. 2026-03-31]. Dostupné z: <https://ahefner.livejournal.com/11670.html>.
13. *APU Mixer - NESdev Wiki* [online]. [cit. 2026-03-31]. Dostupné z: https://www.nesdev.org/wiki/APU_Mixer.
14. *RP2350 Datasheet: A microcontroller by Raspberry Pi*. 2023. Raspberry Pi Ltd. Dostupné také z: <https://pip-assets.raspberrypi.com/categories/1214-rp2350/documents/RP-008373-DS-2-rp2350-datasheet.pdf?disposition=inline>.
15. DIGITAL DISPLAY WORKING GROUP. *Digital Visual Interface Revision 1.0*. 1999. Dostupné také z: <https://glenwing.github.io/docs/DVI-1.0.pdf>.

16. SD ASSOCIATION. *SD Specifications Part 1 Physical Layer Simplified Specification Version 9.10*. 2023. Dostupné také z: https://www.sdcard.org/downloads/pls/pdf/?p=Part1_Physical_Layer_Simplified_Specification_Ver9.10.jpg&f=Part1PhysicalLayerSimplifiedSpecification10Fin_20231201.pdf.
17. WHITE, Elecia. *Ping Pong Buffers* [online]. [cit. 2026-03-31]. Dostupné z: <https://embedded.fm/blog/2017/3/21/ping-pong-buffers>.
18. *Mesen - Emulation General Wiki* [online]. [cit. 2026-03-31]. Dostupné z: <https://emulation.gametechniki.com/index.php/Mesen>.
19. *Emulator Tests - NESdev Wiki* [online]. [cit. 2026-03-31]. Dostupné z: https://www.nesdev.org/wiki/Emulator_tests.

Seznam výpisů

1	Smyčka emulátoru. CPU provádí určitý počet instrukcí, a pak PPU a APU dohání jeho stav.	38
2	Komunikační rozhraní sběrnice (příklad funkce pro zápis do adresního prostoru).	39
3	Krokovací funkce CPU.	40
4	Funkce pro aktualizaci stavu PPU, implementovaná pomocí catch-up mechanismu.	40
5	Funkce pro vykreslování dlaždicových řádku pozadí.	41
6	Extrakce řádku dlaždice a následná jeho syntézu do 32bitového typu TileRow pomocí LUT.	42
7	Kód pro vykreslení spritu s řešením priority pořadí, Sprite 0 Hit a průhledností.	43
8	Implementace Sprite Evaluation. Funkce připravuje data pro 8 objektů na aktuálním řádku.	44
9	Aktualizace stavu APU.	46
10	Generace vzorků pomocí LUT tabulky, která má předvypočítané hodnoty pomocí lineární aproximace popsané v [5].	47
11	Struktura kódu BIOS (zkráceno)	48
12	Příklad využití komunikačního rozhraní FWX v kódu BIOSu.	48
13	Hlavní smyčka programu firmwaru.	63
14	Inicializace DVI.	64
15	Inicializace a konfigurace PWM.	67
16	Ukládání vzorku do fronty (volného bufferu).	69
17	Zdrojový kód PIO programu pro asynchronní čtení ovladačů.	70
18	Měření průměrného času, potřebného na emulaci jednoho snímku.	73

Seznam obrázků

1.1	Znázornění catch-up mechanismu na cyklové ose.	13
1.2	Znázornění dohánění na požadavek.	14
1.3	Příklad mapované paměti: adresový prostor CPU systému NES.	16
1.4	Ukázka dlaždicové grafiky v systému NES: zatímco mřížka pozadí je fixní, mřížka jednotlivých spritů se může volně pohybovat. Pořízeno z emulátoru Mesen. . . .	18
1.5	Znázornění 4bitového PCM: analogový průběh se dělí na diskrétní body. Zdroj [3].	20
2.1	Indexování do palet pomocí bitových rovin. Zdroj [10].	26
2.2	Hardwarová paleta systému NES. Zdroj [11].	27
2.3	Znázornění scrollingu: šedé obdélníky vyznačují viditelnou část jmenných tabulek, zatímco oblast mezi nimi není hráčem viditelná. Hry předem zapisují do neviditelných oblastí jmenných tabulek další dlaždice mapy. Je zde také patrné zrcadlení jmenných tabulek. Pořízeno z emulátoru Mesen.	28
2.4	Přehled zvukových kanálů APU. Zdroj [6].	29
3.1	Vývojová deska Raspberry Pi Pico 2 s mikrořadičem RP2350.	33
4.1	Koncepční schéma architektury projektu, která je postavena na 3 vrstvách: emulátor, firmware a hardware.	35
6.1	Ukázka hlavního menu BIOS.	47
6.2	Zjednodušené schéma typického průběhu komunikace přes rozhraní FWX.	49
6.3	Detailní stavový diagram procesu odesílání dat z emulovaného programu do emulátoru.	50
6.4	Detailní stavový diagram procesu příjmu návratových dat emulovaným programem.	51
6.5	Rozvržení registru FWXSTAT.	52
6.6	Rozvržení registru FWXCTRL.	53
7.1	Vyrobená základní deska.	56
7.2	Blokové schéma analogové částí audio výstupu.	58
9.1	Koncept kříže na ovladači.	62
9.2	Příprava krytu základní desky pro 3D tisk v programu PrusaSlicer.	62
10.1	Blokové schéma principu dvou DMA kanálů: audio kanál a řídicí kanál.	67

11.1	Program liveness, který dovoluje přepínat jednotlivé bity registrů APU.	72
11.2	Testovací sada 240p-test-mini.	72
11.3	Screenshot debuggeru.	74
12.1	Finální výrobek.	77

Seznam tabulek

1	Přehled použitých technologií, knihoven a nástrojů.	9
2.1	Struktura hlavičky formátu iNES.	22
2.2	Zjednodušená mapa paměťového prostoru CPU	23
2.3	Paměťová mapa PPU.	25
6.1	Přehled komunikačních registrů rozhraní FWX.	51
6.2	Přehled chybových kódů vrácených v registru FWXSTAT.	53
6.3	Přehled příkazů FWX rozhraní. Datové typy argumentů a návratových hodnot jsou uvedeny v závorkách.	55

Seznam příloh

Příloha A – Elektrické schéma základní desky

Příloha B – Návrh desky plošných spojů základní desky

Příloha C – Elektrické schéma ovladače

Příloha D – Návrh desky plošných spojů ovladače

Příloha E – Popis schématu analogové zvukové částí

Příloha F – Podrobný popis funkcí SD karty

Příloha G – Popis vybraných periférií RP2350